

Enhancing YouTube Data Donations With Platform Metadata

Anonymized website and full notebook companion for review

This PDF is generated from the rendered companion website. It includes the main page, the notebook guide, and the six full rendered notebooks while omitting identifying repository and authorship material.

Contents

1. Main Page
2. Notebooks Subpage
3. 01 Clean Donated YouTube Watch Data
4. 02 Clean Donated YouTube Search Data
5. 03 Scrape YouTube Metadata With yt-dlp
6. 04 Link Watch Data To Metadata, Exposure Context, And Problematic-View Labels
7. 05 Estimate Watch Time From Enriched Watch Events
8. 06 Demonstrate Enriched YouTube Exposure Measures

Enhancing YouTube Data Donations With Platform Metadata

This repository demonstrates how donated YouTube account data can be linked with video- and channel-level platform metadata to create **enriched exposure histories**.

The project accompanies the method described in *Enhancing YouTube Data Donations With Platform Metadata*. It shows, in reproducible form, how researchers can move from raw Google Takeout-style files, as obtained via data donations, to event-level indicators of YouTube exposure: what was watched, when it was watched, estimated watch time, whether it followed a search, whether it came from a subscribed channel, whether it was a Short or Long, and whether later metadata collection suggests that a video became unavailable for potentially problematic content-related reasons. For demonstration purposes this repository uses **synthetic mock data**.

Purpose

YouTube watch histories are valuable for media, journalism and platform research because they record concrete viewing behavior over time rather than recall and report biased self-reports, aggregate platform metrics, or metadata by themselves. On their own, however, donated watch histories are sparse.

This project shows how those traces can be enriched by joining them with video-level metadata through the YouTube `video_id`. After enrichment, each watch event can be analyzed not only as a timestamped view, but as part of a wider exposure history containing information about video format, channel context, likely viewing pathway, availability status, and time of day. In aggregate this allows inference not only on when but also on how data donors used the platform and provides additional information on what the people encountered.

The workflow is meant for researchers who want to understand or adapt a data donation pipeline for YouTube-related research, especially in journalism studies, platform accountability research, computational social science, and media-use research.

What This Repository Demonstrates

The repository shows how to:

- clean donated YouTube watch, search, and subscription files;
- extract and standardize YouTube `video_id`s across donated files;
- prepare a video-level metadata table;
- join donated histories and metadata at the event level;
- classify watches as search-associated or subscription-linked where possible;
- distinguish Shorts from Longs using metadata-derived rules;
- classify viewing as daytime or nighttime according to a configurable time window;
- estimate watch time from metadata duration and the next observed watch event;
- attach availability and problematic-view indicators based on metadata/error signals;

The goal is not to reconstruct every individual viewing decision. YouTube does not provide authoritative explanations for why a specific video appeared in a user's history and we have no access to what a given video meant to a given viewer at time of viewing. However, the workflow produces **aggregate exposure indicators** that allow insight into how YouTube use differs across users.

What Enrichment Adds

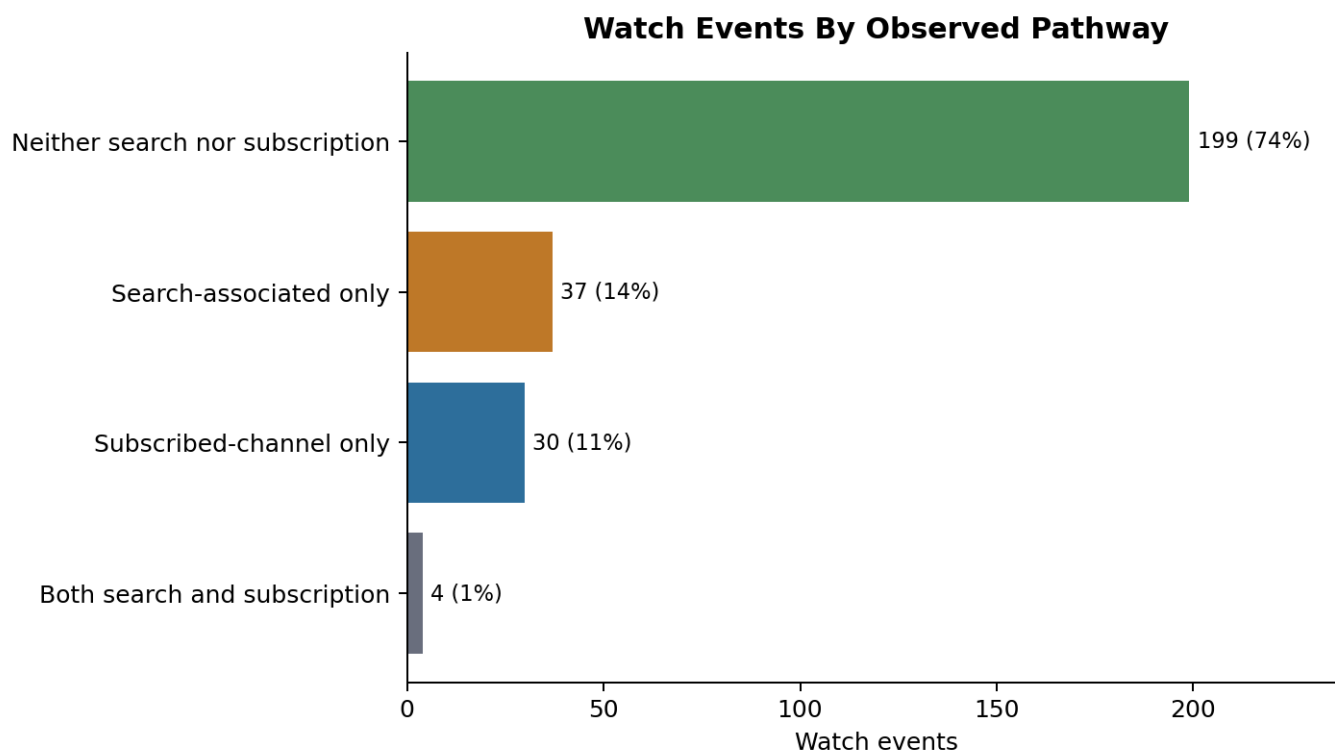
A raw watch-history row tells us that a participant started watching a video at a particular time. After enrichment, the same row can also include:

- video duration;
- channel and channel identifier;
- upload date;
- estimated watch time;
- view, like, and comment metadata;
- language, format, aspect ratio, and other video-level fields;
- whether the video is classified as a Short or Long;
- whether the watch happened during the configured day or night period;
- indication of whether the watch followed an explicit search;
- indication of whether the video came from a channel the user is subscribed to;
- whether a watched video was later removed by YouTube;
- whether the removal was due to potentially problematic content of the video

These data can then be used for user-level assessments like:

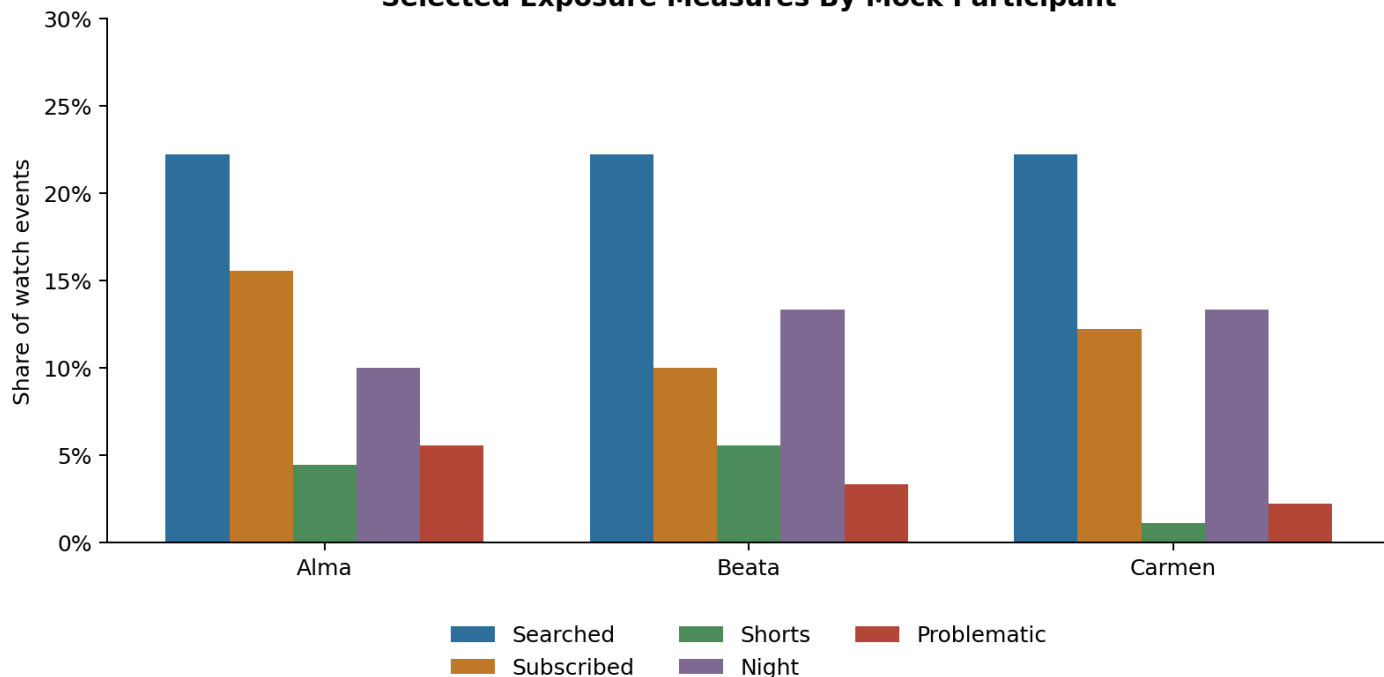
- share of searched or subscribed watches among all watches
- estimated watch time across participants, formats, and routines
- distribution of Shorts/Longs across day/night watches
- frequency of encountering potentially problematic content

These indicators are imperfect but useful. They are strongest when used to describe patterns across many events or participants, rather than to make definitive claims about single views. The figures below exemplify how the data can be used to quantify YouTube usage and identify differences between individual data donors. These examples are illustrative outputs from the synthetic mock data.



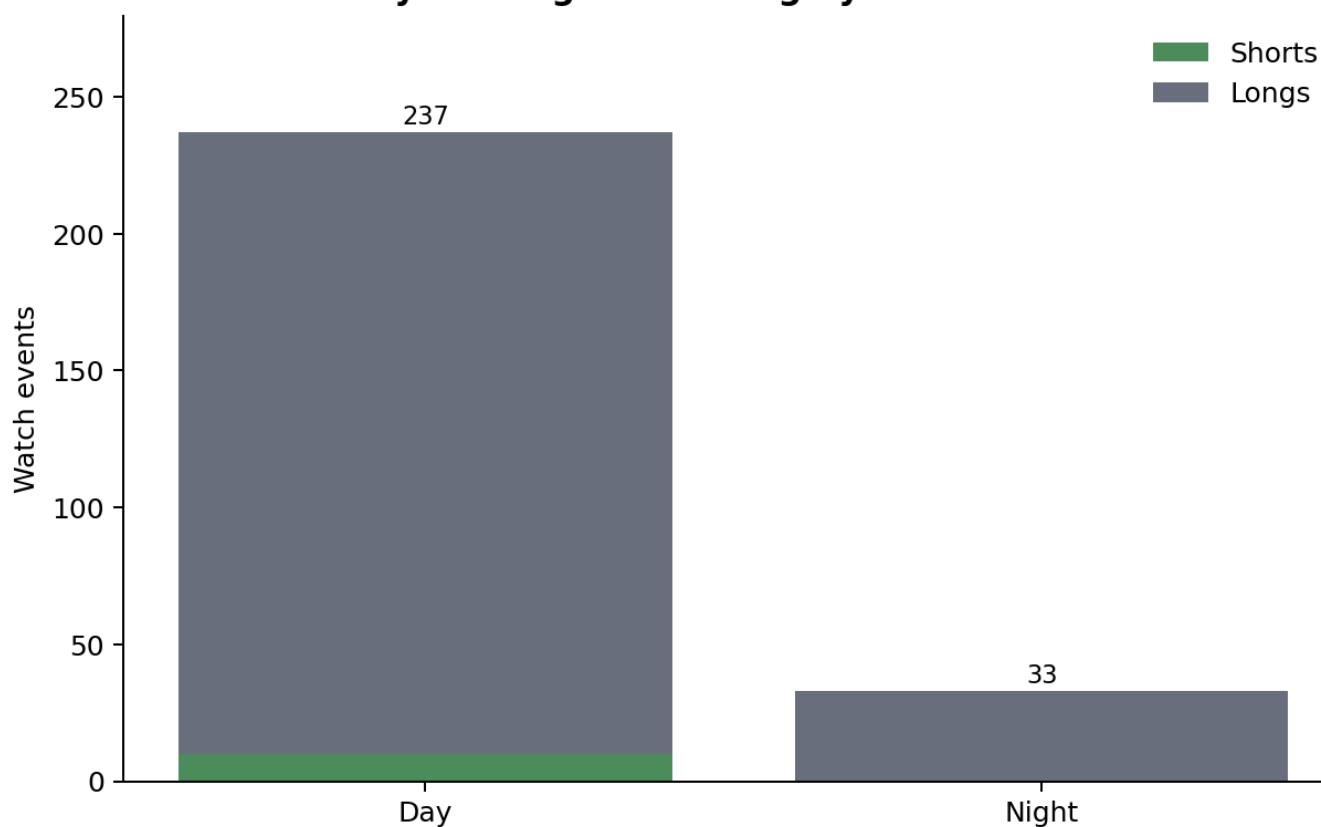
Search and subscription context make it possible to distinguish whether watches followed explicit searches, came from subscribed channels, both, or neither.

Selected Exposure Measures By Mock Participant



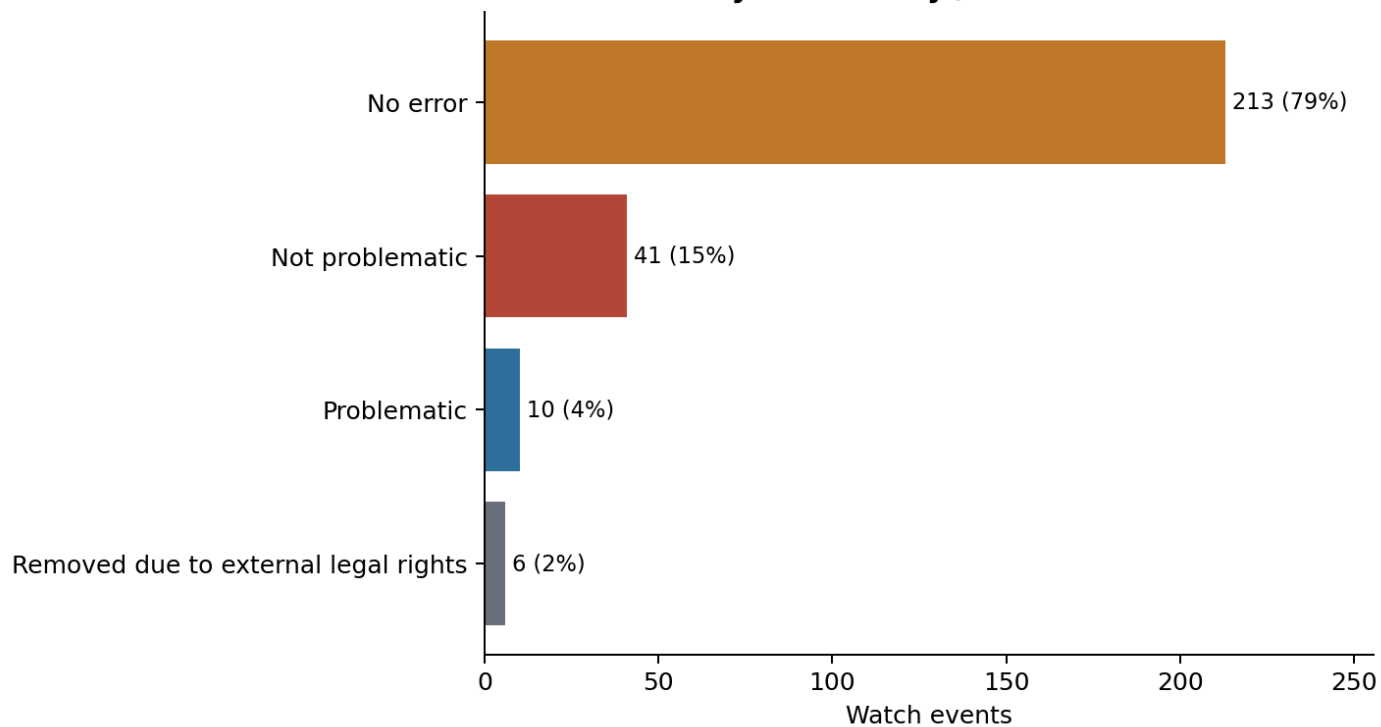
The enriched event table allows participant-level comparison across searched, subscribed, Shorts, nighttime, and problematic-view indicators.

Day And Night Watching By Video Format



Metadata-derived format labels and localized timestamps allow analyses of Shorts/Longs across day and night watching.

Watch Events By Availability / Problematic Assessment

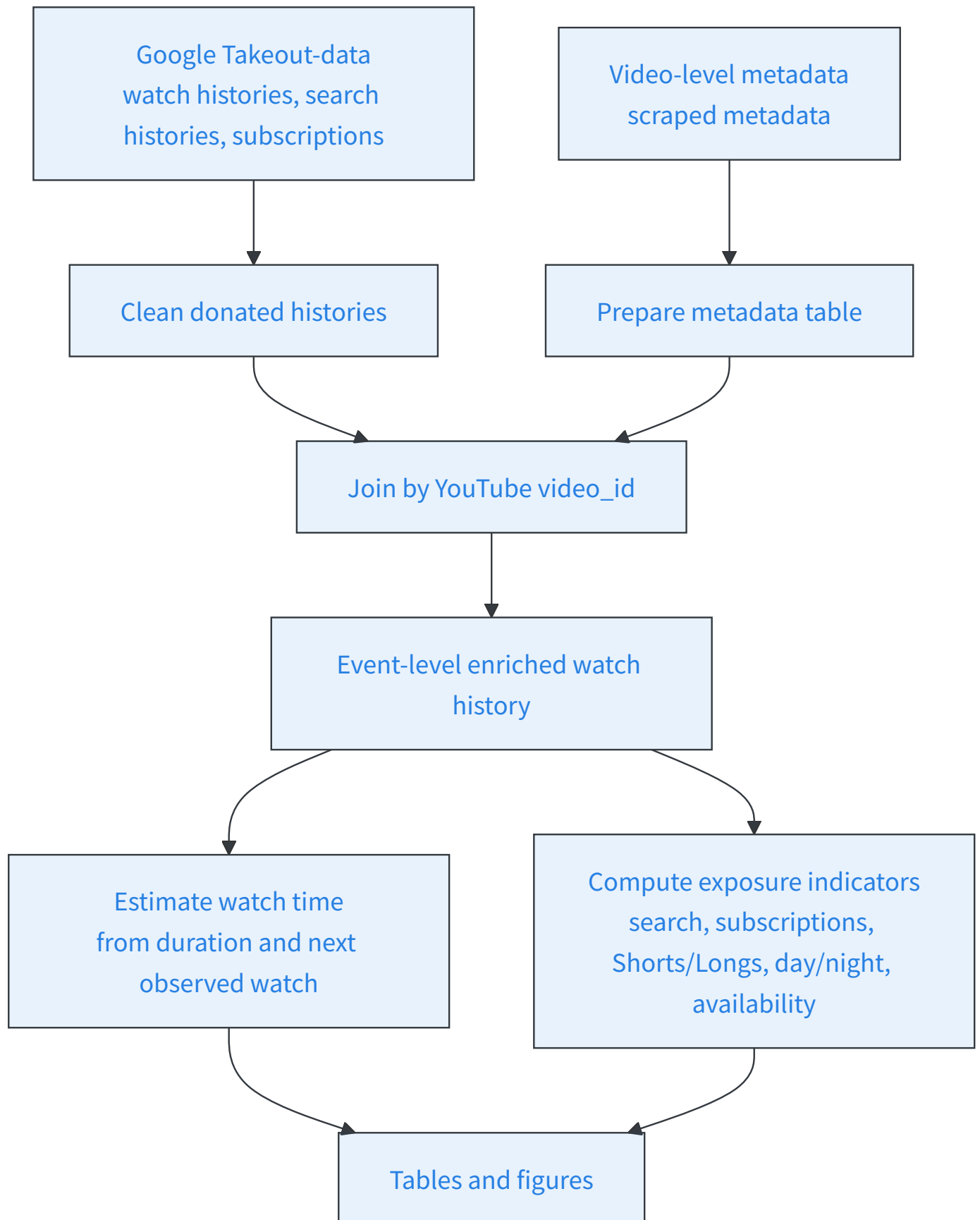


Scraped metadata errors and manual classifications make unavailable or potentially problematic watched videos analyzable at the event level.

Workflow

flowchart TD

```
A["Google Takeout-data<br/>watch histories, search histories, subscriptions"] --> B["Clean donated histories"]
C["Video-level metadata<br/>scraped metadata"] --> D["Prepare metadata table"]
B --> E["Join by YouTube video_id"]
D --> E
E --> F["Event-level enriched watch history"]
F --> G["Estimate watch time<br/>from duration and next observed watch"]
F --> H["Compute exposure indicators<br/>search, subscriptions, Shorts/Longs, day/night, availability"]
G --> I["Tables and figures"]
H --> I
```



Recommended Entry Point

For a quick overview of the finished workflow, start with:

[Notebook 06: Demonstrate enriched exposure measures](#)

This notebook shows the final enriched data structure and produces compact figures that illustrate what the method makes visible.

Then use the [notebook guide](#) to inspect the earlier steps in order: cleaning donated histories, preparing metadata, demonstrating scraping logic, linking records, estimating watch time, and producing the final exposure measures.

Mock Data

The mock dataset contains three fictional donors: Alma, Beata, Carmen. Each donor folder contains small watch-history, search-history, and subscription files with the same broad structure as a YouTube Takeout export. The public identifiers and text fields are synthetic. Video IDs, titles, channel IDs, channel names, search queries, tags, and URLs have been replaced with deterministic fake values.

Legalities of Metadata Collection

[Notebook 03](#) demonstrates the metadata collection logic used in the method. In a real research project, metadata collection requires careful legal, ethical, and institutional assessment. It may also depend on the researcher's access rights, data security setup, and platform access conditions.

One route to obtain research access to YouTube is the Digital Services Act Article 40(12) for research with a systemic risk focus. Access can be requested via this [request records form](#) and requires provision of the scraper's IP and a *product token string* for authentication. Researchers with access can make themselves identifiable to YouTube by using their product token string as `IDENTIFYING_USER_AGENT` when scraping.

An example data-access request link is omitted from this anonymized review copy.

For further support in requesting data access under Article 40 of the Digital Services Act (DSA), the [DSA Collaboratory](#) is highly recommended.

Notebook Guide

The notebooks in this repository are designed as a pipeline. They show how donated YouTube account data can be cleaned, linked to platform metadata, and transformed into event-level exposure indicators.

The public version uses mock Google Takeout-style data and a scrambled metadata fixture. This means readers can inspect and reproduce the workflow without access to private participant donations or real YouTube identifiers.

Recommended Reading Paths

There are two useful ways to read the notebooks.

If you want the **quickest sense of what the method produces**, start with [Notebook 06](#). It shows the enriched event table and the final demonstration figures.

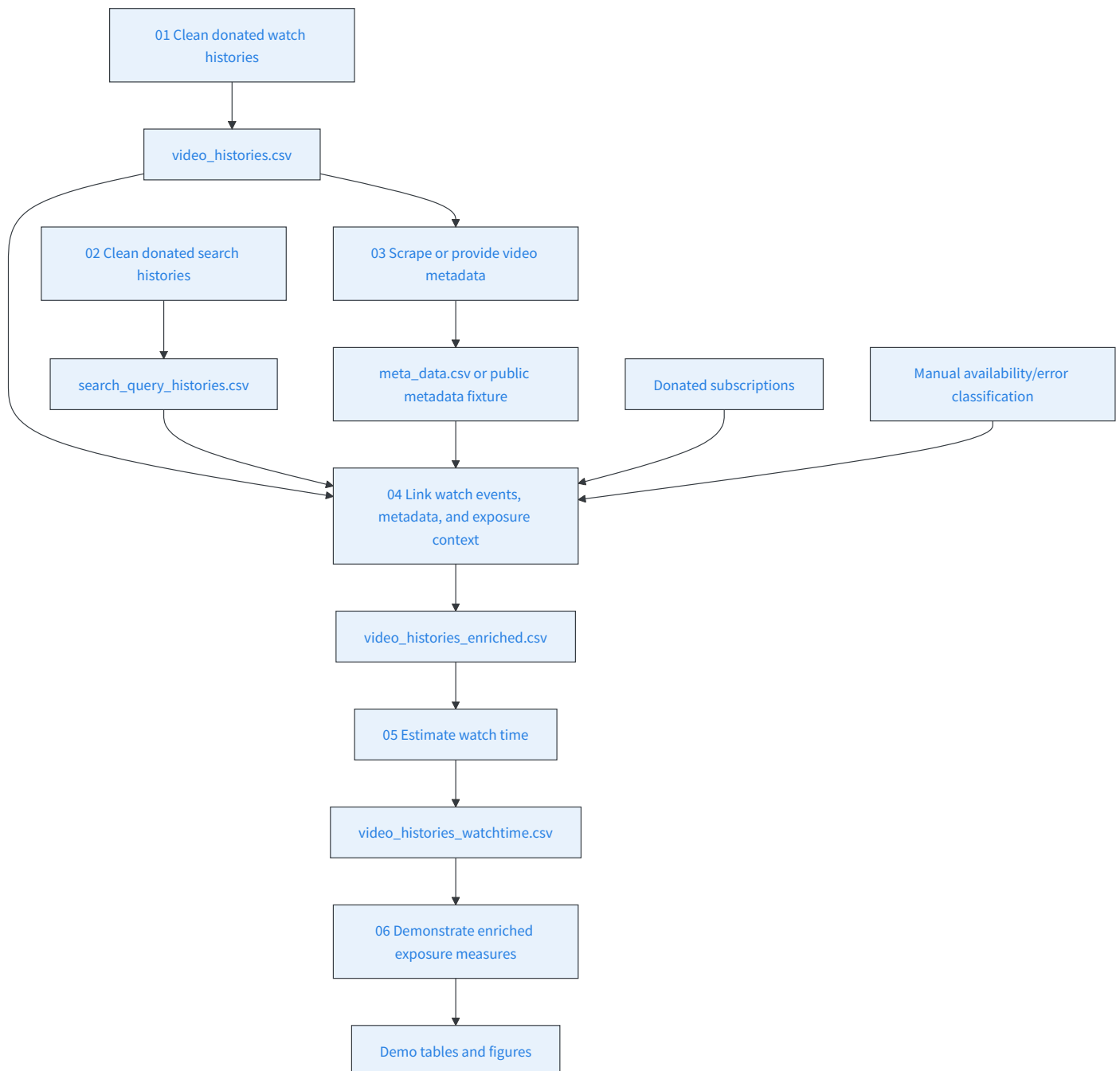
If you want to understand the **full workflow**, read the notebooks in order:

1. clean donated watch histories;
2. clean donated search histories;
3. document metadata collection;
4. link watch events, metadata, subscriptions, searches, and availability signals;
5. estimate watch time from metadata duration and the next observed watch event;
6. summarize the enriched exposure measures.

Pipeline Overview

flowchart TD

```
A["01 Clean donated watch histories"] --> B["video_histories.csv"]
C["02 Clean donated search histories"] --> D["search_query_histories.csv"]
B --> E["03 Scrape or provide video metadata"]
E --> F["meta_data.csv or public metadata fixture"]
B --> G["04 Link watch events, metadata, and exposure context"]
D --> G
F --> G
H["Donated subscriptions"] --> G
I["Manual availability/error classification"] --> G
G --> J["video_histories_enriched.csv"]
J --> K["05 Estimate watch time"]
K --> L["video_histories_watchtime.csv"]
L --> M["06 Demonstrate enriched exposure measures"]
M --> N["Demo tables and figures"]
```



Notebook Summary

Notebook	Main role	Main output
01 Clean Donated Watch Data	Turns raw Takeout-style watch histories into validated video-watch events.	<code>outputs/tables/video_histories.csv</code>
02 Clean Donated Search Data	Turns raw Takeout-style search histories into genuine YouTube search-query events.	<code>outputs/tables/search_query_histories.csv</code>
03 Scrape YouTube Metadata	Documents how video-level metadata can be collected with <code>yt_dlp</code> .	<code>outputs/tables/meta_data.csv</code>
04 Link Watch Data, Metadata, And Exposure Context	Joins watch events with metadata, search, subscription, time-of-day, format, and availability indicators.	<code>outputs/tables/video_histories_enriched.csv</code>
05 Estimate Watch Time	Estimates event-level watch time from metadata duration and the next observed watch event.	<code>outputs/tables/video_histories_watchtime.csv</code>
06 Demonstrate Enriched Exposure Measures	Shows what the enriched data makes visible and produces demonstration figures.	Demo tables and figures in <code>outputs/</code>

01 Clean Donated Watch Data

[Open notebook 01](#)

This notebook is the first cleaning step. It reads mock Google Takeout-style YouTube watch-history files from `data/mock_takeout`, combines donor files, standardizes columns, parses timestamps, extracts YouTube `video_id`s, and separates validated video watches from ads, YouTube Music rows, and other non-video rows.

It also cleans multilingual watch-action prefixes so that titles can be interpreted more consistently across Takeout exports.

The main handoff is:

- `outputs/tables/video_histories.csv`

This table keeps one row per retained YouTube video watch event. It contains the participant ID, timestamp, watched title, cleaned title, URL, extracted `video_id`, and channel information where available.

The notebook also writes side tables for auditing:

- `outputs/tables/watch_histories.csv`
- `outputs/tables/music_histories.csv`
- `outputs/tables/ad_watch_histories.csv`
- `outputs/tables/other_histories.csv`
- `outputs/tables/title_prefix_diagnostics.csv`

These side tables are useful because they show what was retained, what was excluded, and which localized title prefixes were recognized.

02 Clean Donated Search Data

[Open notebook 02](#)

This notebook cleans donated YouTube search-history files. It reads mock Takeout-style search histories, combines donor files, standardizes columns, parses timestamps, decodes search queries from URLs, and identifies rows that represent genuine YouTube searches.

The main handoff is:

- `outputs/tables/search_query_histories.csv`

This table is used later to determine whether a watch event occurred shortly after a participant searched on YouTube.

The notebook also writes:

- `outputs/tables/search_histories.csv`

The distinction matters because not every row in a Takeout search-history file is necessarily useful for the exposure pathway indicator. The later workflow relies only on rows classified as genuine search-query events.

03 Scrape YouTube Metadata

[Open notebook 03](#)

This notebook documents the metadata collection step. It reads unique `video_id`s from `outputs/tables/video_histories.csv` and shows how `yt_dlp` can be used to request public YouTube metadata for each video.

The intended output of a real metadata run is:

- `outputs/tables/meta_data.csv`

Each row represents one unique YouTube video ID. If metadata can be retrieved, the row contains video-level fields such as title, channel, duration, upload date, format information, popularity metrics, language, subtitle availability, and related metadata. If metadata cannot be retrieved, the row is still kept and the raw `yt_dlp` error message is stored.

Warning

This notebook is documented but not executed in the public site. The public repository contains scrambled fake video IDs, which preserve the shape of the workflow but cannot be scraped from YouTube.

To run this notebook in a real project, researchers would need authorized real YouTube IDs, appropriate legal and institutional permissions, and a registered identifying User-Agent token or equivalent project identifier. The notebook stops by default until `YOUR_REGISTERED_TOKEN` is replaced.

For public reproduction, later notebooks can use the scrambled fixture:

- `data/demo_metadata/meta_data_public.csv`

This allows readers to understand the linking workflow without running live YouTube metadata collection.

04 Link Watch Data, Metadata, And Exposure Context

[Open notebook 04](#)

This is the main enrichment notebook. It joins cleaned watch events to video-level metadata and adds the event-level indicators that turn raw watch histories into enriched exposure histories.

It reads:

- `outputs/tables/video_histories.csv`
- `outputs/tables/search_query_histories.csv`

- `outputs/tables/meta_data.csv`, if available
- `data/demo_metadata/meta_data_public.csv`, as the public fallback metadata fixture
- donated subscription files from `data/mock_takeout`
- `data/manual/unavailable_content_classification_filled.csv`

It writes the main enriched event table:

- `outputs/tables/video_histories_enriched.csv`

Each row is still one donated watch event, but now the event also carries metadata, linkage status, local-time helper fields, Shorts/Longs classification, day/night classification, search-associated markers, subscribed-channel markers, and availability/problematic-view labels.

The notebook also writes audit and helper files:

- `outputs/tables/metadata_linkage_report.csv`
- `outputs/tables/subscription_histories.csv`
- `outputs/tables/exposure_linkage_report.csv`
- `outputs/tables/error_types_by_video.csv`

Several methodological choices are made transparent in this notebook:

- the day/night boundary is configurable;
- the mock project uses `Europe/Copenhagen` as the default timezone;
- Shorts are classified from metadata using duration and aspect-ratio logic;
- search-associated views are marked when a participant search occurred 1 to 55 seconds before the watch event;
- subscription-linked views are identified by comparing metadata channel IDs with donated subscription channel IDs;
- raw metadata errors are preserved before being generalized and manually classified.

The notebook keeps the unit of analysis at the watch-event level. No donated watch event is dropped simply because metadata is missing or returned an error.

05 Estimate Watch Time

[Open notebook 05](#)

This notebook estimates watch time from the enriched event table created by notebook 04. It uses the original workflow's core idea: if a participant starts another video before the current video's metadata duration has elapsed, the remaining duration is treated as skipped.

It reads:

- `outputs/tables/video_histories_enriched.csv`

It writes:

- `outputs/tables/video_histories_watchtime.csv`
- `outputs/tables/watchtime_summary.csv`
- `outputs/tables/watchtime_diagnostics.csv`

The event-level output keeps one row per watch event and adds helper fields such as the next observed watch time, seconds until the next watch, estimated watch seconds, and a watch-time status flag. The summary collapses those estimates across the full mock dataset by participant, Shorts/Longs, and day/night.

This estimate should not be interpreted as exact viewing completion. It is a transparent approximation based on observed start times and video duration metadata.

06 Demonstrate Enriched Exposure Measures

[Open notebook 06](#)

This is the showcase notebook. It starts from the enriched event table produced by notebook 04 and demonstrates what the method makes visible.

It reads:

- `outputs/tables/video_histories_enriched.csv`
- `outputs/tables/video_histories_watchtime.csv`, when available
- `outputs/tables/exposure_linkage_report.csv`, when available

It writes:

- `outputs/tables/demo_enriched_event_examples.csv`
- `outputs/tables/demo_measure_summary.csv`
- `outputs/figures/demo_pathway_mix.png`
- `outputs/figures/demo_participant_measure_shares.png`
- `outputs/figures/demo_daynight_shorts.png`
- `outputs/figures/demo_problematic_views.png`

The notebook first displays selected enriched watch events. These examples show how a single donated watch event can now be read together with metadata fields, pathway indicators, format labels, day/night labels, and availability/problematic-view signals.

It then creates four compact figures:

1. **Pathway mix**

Shows how many watched events are search-associated, subscription-linked, both, or neither.

2. **Participant-level measure shares**

Summarizes exposure indicators by fictional donor in the mock dataset.

3. **Day/night and Shorts/Longs**

Combines local viewing time with metadata-derived format labels.

4. **Problematic and unavailable signals**

Demonstrates how later metadata errors and manual classification can be summarized as availability or problematic-view indicators.

01 Clean Donated YouTube Watch Data

This notebook demonstrates the first step in the data-donation workflow: turning raw Google Takeout-style YouTube watch-history files into clean event-level tables.

The notebook uses mock donor data stored in `data/mock_takeout`. The same logic can be adapted to confidential GDPR data donations by changing the input path and adding project-specific consent, exclusion, and participant metadata rules.

What This Notebook Produces

The notebook writes several CSV files to `outputs/tables`.

- `watch_histories.csv`: all donated watch-history rows combined and standardized.
- `video_histories.csv`: the validated video-watch table used by later notebooks.
- `music_histories.csv`: rows identified as YouTube Music.
- `ad_watch_histories.csv`: rows marked as Google Ads.
- `other_histories.csv`: rows that are neither validated videos, music, nor ads.
- `title_prefix_diagnostics.csv`: counts showing which localized watch-action prefixes were recognized.

The main analysis handoff is `video_histories.csv`, where each row represents one retained YouTube video watch event with a participant ID, timestamp, URL, `video_id`, cleaned title, and channel information when available.

```
from pathlib import Path
import re

import pandas as pd
```

Locate The Project And Input Data

The notebook is designed to run from the Quarto project root or from the `scripts` folder. The helper below walks upward from the current working directory until it finds the Quarto project and the mock Takeout folder.

```
def find_project_root(start=None):
    start = Path.cwd() if start is None else Path(start)
    for candidate in [start, *start.parents]:
        has_quarto_project = (candidate / "_quarto.yml").exists()
        has_mock_data = (candidate / "data" / "mock_takeout").exists()
        if has_quarto_project and has_mock_data:
            return candidate
    raise FileNotFoundError(
        "Could not find the Quarto project root. Run this notebook from "
        "inside youtube_donation_dsa_method."
    )

PROJECT_ROOT = find_project_root()
MOCK_TAKEOUT_DIR = PROJECT_ROOT / "data" / "mock_takeout"
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "tables"

WATCH_HISTORIES_PATH = OUTPUT_DIR / "watch_histories.csv"
VIDEO_HISTORIES_PATH = OUTPUT_DIR / "video_histories.csv"
MUSIC_HISTORIES_PATH = OUTPUT_DIR / "music_histories.csv"
AD_WATCH_HISTORIES_PATH = OUTPUT_DIR / "ad_watch_histories.csv"
OTHER_HISTORIES_PATH = OUTPUT_DIR / "other_histories.csv"
TITLE_PREFIX_DIAGNOSTICS_PATH = OUTPUT_DIR / "title_prefix_diagnostics.csv"

print(f"Project folder: {PROJECT_ROOT.name}")
print(f"Mock data folder: {MOCK_TAKEOUT_DIR.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Output folder: {OUTPUT_DIR.relative_to(PROJECT_ROOT).as_posix()}")
```

Find Donated Watch-History Files

Google Takeout stores YouTube watch history at [YouTube and YouTube Music/history/watch-history.json](#). In this mock dataset, each donor has one folder, so the donor folder name becomes the participant identifier.

```
watch_files = sorted(
    MOCK_TAKEOUT_DIR.glob("*/YouTube and YouTube Music/history/watch-history.json")
)

watch_file_index = pd.DataFrame(
    {
        "Participant ID": [path.relative_to(MOCK_TAKEOUT_DIR).parts[0] for path in watch_files],
        "watch_history_file": [path.relative_to(PROJECT_ROOT).as_posix() for path in watch_files],
    }
)

watch_file_index
```

Load And Combine Watch Histories

Each JSON file is loaded into a dataframe, annotated with [Participant ID](#), and then combined into one raw watch-history table. At this point, the table still contains videos, ads, music rows, and any other Takeout row types that may appear.

```
def load_watch_history(path):
    participant_id = path.relative_to(MOCK_TAKEOUT_DIR).parts[0]
    data = pd.read_json(path)
    data["Participant ID"] = participant_id
    return data

watch_histories = pd.concat(
    [load_watch_history(path) for path in watch_files],
    ignore_index=True,
)

watch_histories.shape
```

Standardize Columns

The original Takeout fields are useful but not always named for analysis. We rename `title` to `watched_title` and `titleUrl` to `url`, then keep the columns needed for cleaning, auditing, and side-table exports.

```
watch_histories = watch_histories.rename(
    columns={
        "title": "watched_title",
        "titleUrl": "url",
    }
)

core_columns = [
    "Participant ID",
    "time",
    "header",
    "watched_title",
    "url",
    "details",
    "subtitles",
]

watch_histories = watch_histories.reindex(columns=core_columns)
watch_histories.head()
```

Parse Timestamps

Takeout timestamps are ISO-formatted UTC strings. Some rows include milliseconds and some do not, so we use pandas' ISO8601 parser explicitly.

```
watch_histories["time"] = pd.to_datetime(
    watch_histories["time"],
    format="ISO8601",
    utc=True,
)

watch_histories["time"].agg(["min", "max"])
```

Extract YouTube Video IDs

The `video_id` is the stable join key for linking donated watch events to scraped or platform-provided metadata. We extract it only from standard YouTube watch URLs with an 11-character ID.

```
YOUTUBE_WATCH_URL = re.compile(
    r"https://www\.youtube\.com/watch?v=([A-Za-z0-9_-]{11})$"
)

def extract_video_id(url):
    if not isinstance(url, str):
        return pd.NA
    match = YOUTUBE_WATCH_URL.match(url)
    return match.group(1) if match else pd.NA

watch_histories["video_id"] = watch_histories["url"].apply(extract_video_id)
watch_histories["has_valid_video_id"] = watch_histories["video_id"].notna()

watch_histories[["watched_title", "url", "video_id"]].head()
```

Handle `details` And Classify Rows

This step follows the validated selection mechanic from the original workflow: a retained video-watch row must have a standard watch URL, an 11-character `video_id`, and no `details` value. Rows with `details` are excluded from `video_histories` because in the original data this field captured ads and other non-standard watch-history records.

We still detect Google Ads explicitly and save them separately, so readers can inspect what was excluded.

```
def details_missing(value):
    # This mirrors the original `details.isna()` selection rule.
    # Lists or dictionaries mean Takeout supplied a details field, so the row is not a clean video watch.
    if isinstance(value, (list, dict)):
        return False
    return bool(pd.isna(value))

def has_google_ads_marker(details):
    if not isinstance(details, list):
        return False
    return any(
        isinstance(item, dict) and item.get("name") == "From Google Ads"
        for item in details
    )

watch_histories["details_missing"] = watch_histories["details"].apply(details_missing)
watch_histories["has_google_ads_marker"] = watch_histories["details"].apply(has_google_ads_marker)
watch_histories["is_youtube_music"] = watch_histories["header"].eq("YouTube Music")

video_mask = (
    watch_histories["has_valid_video_id"]
    & watch_histories["details_missing"]
    & ~watch_histories["is_youtube_music"]
)
music_mask = watch_histories["is_youtube_music"]
ad_watch_mask = watch_histories["has_google_ads_marker"]
other_mask = ~(video_mask | music_mask | ad_watch_mask)

def classify_watch_row(row):
    if row["is_youtube_music"]:
        return "youtube_music"
    if row["has_valid_video_id"] and row["details_missing"]:
        return "video_watch"
    if row["has_google_ads_marker"]:
        return "ad_watch"
    if row["has_valid_video_id"]:
        return "video_with_details"
    return "other_watch_row"

watch_histories["watch_row_type"] = watch_histories.apply(classify_watch_row, axis=1)

watch_histories["watch_row_type"].value_counts()
```

Extract Channel Fields

Takeout often stores channel information inside the nested `subtitles` field. We extract the first channel name and URL when available. Later notebooks can use these fields to check subscription-linked exposure.

```
def first_subtitle_field(subtitles, field):
    if not isinstance(subtitles, list) or not subtitles:
        return pd.NA
    first = subtitles[0]
    if not isinstance(first, dict):
        return pd.NA
    return first.get(field, pd.NA)

def extract_channel_id(channel_url):
    if not isinstance(channel_url, str):
        return pd.NA
    match = re.match(r"^https?://www\.youtube\.com/channel/([A-Za-z0-9_-]+)$", channel_url)
    return match.group(1) if match else pd.NA

watch_histories["channel_title"] = watch_histories["subtitles"].apply(
    lambda subtitles: first_subtitle_field(subtitles, "name")
)
watch_histories["channel_url"] = watch_histories["subtitles"].apply(
    lambda subtitles: first_subtitle_field(subtitles, "url")
)
watch_histories["channel_id"] = watch_histories["channel_url"].apply(extract_channel_id)

watch_histories[["channel_title", "channel_url", "channel_id"]].head()
```

Clean Multilingual Watch Titles

Google localizes the action prefix in Takeout titles according to the donor account language. The video title might start with `Watched`, `Så`, `Hai guardato`, `Просмотрено видео`, or another localized phrase.

We therefore keep a small, explicit lookup table of known watch-action prefixes from the original workflow. The notebook removes only known prefixes, records which prefix was removed, and creates a diagnostic for titles where no known prefix was recognized. This avoids silently mangling titles when a new account language appears.

```
WATCH_ACTION_PREFIXES = pd.DataFrame(  
    [  
        {"language_hint": "English", "prefix": "Watched "},  
        {"language_hint": "Danish/Norwegian", "prefix": "Så "},  
        {"language_hint": "Italian", "prefix": "Hai guardato "},  
        {"language_hint": "Russian", "prefix": "Просмотрено видео "},  
        {"language_hint": "Polish", "prefix": "Obejrzano: "},  
    ]  
)  
  
prefix_lookup = dict(  
    zip(WATCH_ACTION_PREFIXES["prefix"], WATCH_ACTION_PREFIXES["language_hint"])  
)  
WATCH_PREFIX_PATTERN = re.compile(  
    "^(" + "|".join(re.escape(prefix) for prefix in sorted(prefix_lookup, key=len, reverse=True)) + ")"  
)  
NON_WORD_SYMBOLS = re.compile(r"^[^\w\s\u0400-\u04FF]")  
  
WATCH_ACTION_PREFIXES
```

```

def split_known_watch_prefix(title):
    if not isinstance(title, str):
        return pd.Series(
            {
                "watch_action_prefix": pd.NA,
                "watch_action_language_hint": pd.NA,
                "prefix_removed": False,
                "title_without_watch_prefix": pd.NA,
            }
        )

    match = WATCH_PREFIX_PATTERN.match(title)
    if not match:
        return pd.Series(
            {
                "watch_action_prefix": pd.NA,
                "watch_action_language_hint": pd.NA,
                "prefix_removed": False,
                "title_without_watch_prefix": title,
            }
        )

    prefix = match.group(1)
    return pd.Series(
        {
            "watch_action_prefix": prefix.strip(),
            "watch_action_language_hint": prefix_lookup[prefix],
            "prefix_removed": True,
            "title_without_watch_prefix": title[match.end() :].strip(),
        }
    )

def clean_title_text(title):
    if not isinstance(title, str):
        return pd.NA
    without_symbols = NON_WORD_SYMBOLS.sub("", title)
    return re.sub(r"\s+", " ", without_symbols).strip()

prefix_columns = watch_histories["watched_title"].apply(split_known_watch_prefix)
watch_histories[
    [
        "watch_action_prefix",
        "watch_action_language_hint",
        "prefix_removed",
        "title_without_watch_prefix",
    ]
] = prefix_columns
watch_histories["clean_title"] = watch_histories["title_without_watch_prefix"].apply(clean_title_text)

watch_histories[
    ["watched_title", "watch_action_prefix", "watch_action_language_hint", "clean_title"]
].head()

```

Prefix Diagnostics

The prefix diagnostic is a guardrail for real donation projects. If a participant's Takeout export uses an account language that is not in `WATCH_ACTION_PREFIXES`, the title will remain usable, but the action phrase will not be removed. Those cases should be inspected and the prefix list should be extended deliberately.

```

prefix_diagnostics = (
    watch_histories.assign(
        watch_action_prefix=watch_histories["watch_action_prefix"].fillna("UNRECOGNIZED"),
        watch_action_language_hint=watch_histories["watch_action_language_hint"].fillna("UNRECOGNIZED"),
    )
    .groupby(["watch_action_prefix", "watch_action_language_hint"], dropna=False)
    .size()
    .reset_index(name="rows")
    .sort_values("rows", ascending=False)
    .reset_index(drop=True)
)

prefix_diagnostics

```

```

unrecognized_prefix_mask = watch_histories["has_valid_video_id"] & ~watch_histories["prefix_removed"]

unrecognized_title_prefix_examples = (
    watch_histories.loc[
        unrecognized_prefix_mask,
        ["Participant ID", "watched_title", "url"],
    ]
    .assign(
        possible_prefix=lambda df: df["watched_title"].str.extract(r"^(\S+(?:\s+\S+){0,2})")[0]
    )
    .head(20)
)

unrecognized_title_prefix_examples

```

Create Clean And Side Tables

`video_histories` is the narrow analysis table. It uses the validated rule from above: standard video URL, valid `video_id`, and missing `details`.

The side tables preserve rows that are analytically interesting but excluded from ordinary video exposure: YouTube Music, Google Ads, and other unclassified rows.

```

watch_export_columns = [
    "Participant ID",
    "time",
    "header",
    "watch_row_type",
    "watched_title",
    "title_without_watch_prefix",
    "clean_title",
    "watch_action_prefix",
    "watch_action_language_hint",
    "prefix_removed",
    "url",
    "video_id",
    "has_valid_video_id",
    "details_missing",
    "has_google_ads_marker",
    "is_youtube_music",
    "channel_title",
    "channel_url",
    "channel_id",
    "details",
]

video_columns = [
    "Participant ID",
    "time",
    "watched_title",
    "clean_title",
    "watch_action_prefix",
    "watch_action_language_hint",
    "url",
    "video_id",
    "channel_title",
    "channel_url",
    "channel_id",
]

watch_histories_export = (
    watch_histories.loc[:, watch_export_columns]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)

video_histories = (
    watch_histories.loc[video_mask, video_columns]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)

music_histories = (
    watch_histories.loc[music_mask, watch_export_columns]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)

ad_watch_histories = (
    watch_histories.loc[ad_watch_mask, watch_export_columns]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)

other_histories = (
    watch_histories.loc[other_mask, watch_export_columns]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)

video_histories.head()

```

Diagnostics

These checks document what happened during cleaning. For the mock dataset, we expect 300 raw watch-history rows, 270 retained video watches, and 30 ad rows. Music and other rows are included in the workflow even when this mock dataset has none.

```

diagnostics = {
    "donor_files_loaded": len(watch_files),
    "raw_watch_rows": len(watch_histories),
    "combined_watch_history_rows": len(watch_histories_export),
    "video_history_rows": len(video_histories),
    "music_history_rows": len(music_histories),
    "ad_watch_history_rows": len(ad_watch_histories),
    "other_history_rows": len(other_histories),
    "invalid_or_missing_video_id_rows": int(watch_histories["video_id"].isna().sum()),
    "unrecognized_video_prefix_rows": int(unrecognized_prefix_mask.sum()),
    "time_min": watch_histories["time"].min(),
    "time_max": watch_histories["time"].max(),
}

```

```
pd.Series(diagnostics)
```

```

pd.DataFrame(
    {
        "all_watch_rows": watch_histories.groupby("Participant ID").size(),
        "video_rows": video_histories.groupby("Participant ID").size(),
        "ad_rows": ad_watch_histories.groupby("Participant ID").size(),
    }
).fillna(0).astype(int)

```

```
watch_histories["watch_row_type"].value_counts().rename_axis("watch_row_type").to_frame("rows")
```

Save Outputs

The combined `watch_histories.csv` keeps the full cleaned row universe. The narrower `video_histories.csv` is the handoff table for the next step: joining donated watch events to scraped or platform-provided metadata by `video_id`.

```

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

watch_histories_export.to_csv(WATCH_HISTORIES_PATH, index=False)
video_histories.to_csv(VIDEO_HISTORIES_PATH, index=False)
music_histories.to_csv(MUSIC_HISTORIES_PATH, index=False)
ad_watch_histories.to_csv(AD_WATCH_HISTORIES_PATH, index=False)
other_histories.to_csv(OTHER_HISTORIES_PATH, index=False)
prefix_diagnostics.to_csv(TITLE_PREFIX_DIAGNOSTICS_PATH, index=False)

saved_outputs = pd.DataFrame(
    {
        "table": [
            "watch_histories",
            "video_histories",
            "music_histories",
            "ad_watch_histories",
            "other_histories",
            "title_prefix_diagnostics",
        ],
        "path": [
            WATCH_HISTORIES_PATH,
            VIDEO_HISTORIES_PATH,
            MUSIC_HISTORIES_PATH,
            AD_WATCH_HISTORIES_PATH,
            OTHER_HISTORIES_PATH,
            TITLE_PREFIX_DIAGNOSTICS_PATH,
        ],
    }
)
saved_outputs["path"] = saved_outputs["path"].apply(
    lambda output_path: output_path.relative_to(PROJECT_ROOT).as_posix()
)

saved_outputs

```

Result

The workflow now has both a full combined watch-history table and a validated video-watch table. The central join key for later enrichment is `video_id`, while the side tables and prefix diagnostics make the cleaning decisions auditable.

Stored Table Previews

The cells below show the first five rows of each dataframe saved as a CSV. Empty previews are expected when the mock data do not contain that row type.

```
stored_watch_tables = {
    "watch_histories.csv": watch_histories_export,
    "video_histories.csv": video_histories,
    "music_histories.csv": music_histories,
    "ad_watch_histories.csv": ad_watch_histories,
    "other_histories.csv": other_histories,
    "title_prefix_diagnostics.csv": prefix_diagnostics,
}

for file_name, dataframe in stored_watch_tables.items():
    print(f"{file_name} ({len(dataframe)} rows)")
    display(dataframe.head(5))
```

02 Clean Donated YouTube Search Data

This notebook demonstrates the second cleaning step in the data-donation workflow: turning raw Google Takeout-style YouTube search-history files into a combined search-history table and a narrower table of genuine YouTube search queries.

The notebook uses mock donor data stored in `data/mock_takeout`. The same logic can be adapted to confidential GDPR data donations by changing the input path and adding project-specific consent, exclusion, and participant metadata rules.

What This Notebook Produces

The notebook writes two CSV files to `outputs/tables`.

- `search_histories.csv`: all donated search-history rows combined and standardized.
- `search_query_histories.csv`: only rows classified as genuine YouTube search queries.

The second table is the handoff for later notebooks that estimate whether watched videos occurred shortly after a participant searched on YouTube.

```
from pathlib import Path
import re
from urllib.parse import parse_qs, urlparse

import pandas as pd
```

Locate The Project And Input Data

The notebook is designed to run from the Quarto project root or from the `scripts` folder. The helper below walks upward from the current working directory until it finds the Quarto project and the mock Takeout folder.

```
def find_project_root(start=None):
    start = Path.cwd() if start is None else Path(start)
    for candidate in [start, *start.parents]:
        has_quarto_project = (candidate / "_quarto.yml").exists()
        has_mock_data = (candidate / "data" / "mock_takeout").exists()
        if has_quarto_project and has_mock_data:
            return candidate
    raise FileNotFoundError(
        "Could not find the Quarto project root. Run this notebook from "
        "inside youtube_donation_dsa_method."
    )

PROJECT_ROOT = find_project_root()
MOCK_TAKEOUT_DIR = PROJECT_ROOT / "data" / "mock_takeout"
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "tables"

SEARCH_HISTORIES_PATH = OUTPUT_DIR / "search_histories.csv"
SEARCH_QUERY_HISTORIES_PATH = OUTPUT_DIR / "search_query_histories.csv"

print(f"Project folder: {PROJECT_ROOT.name}")
print(f"Mock data folder: {MOCK_TAKEOUT_DIR.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Output folder: {OUTPUT_DIR.relative_to(PROJECT_ROOT).as_posix()}")
```

Find Donated Search-History Files

Google Takeout stores YouTube search history at `YouTube and YouTube Music/history/search-history.json`. In this mock dataset, each donor has one folder, so the donor folder name becomes the participant identifier.

```
search_files = sorted(
    MOCK_TAKEOUT_DIR.glob("*/YouTube and YouTube Music/history/search-history.json")
)

search_file_index = pd.DataFrame(
    {
        "Participant ID": [path.relative_to(MOCK_TAKEOUT_DIR).parts[0] for path in search_files],
        "search_history_file": [path.relative_to(PROJECT_ROOT).as_posix() for path in search_files],
    }
)

search_file_index
```

Load And Combine Search Histories

Each JSON file is loaded into a dataframe, annotated with `Participant ID`, and then combined into one raw search-history table. At this point, the table can include genuine search queries as well as other rows that appear in the search-history export.

```
def load_search_history(path):
    participant_id = path.relative_to(MOCK_TAKEOUT_DIR).parts[0]
    data = pd.read_json(path)
    data["Participant ID"] = participant_id
    return data

search_histories = pd.concat(
    [load_search_history(path) for path in search_files],
    ignore_index=True,
)

search_histories.shape
```

Standardize Columns

The original Takeout fields are useful but not always named for analysis. We rename `title` to `search_title` and `titleUrl` to `url`. We keep `details` and `description` in the combined table so non-query rows remain inspectable without assigning them stronger labels than the data support.

```
search_histories = search_histories.rename(
    columns={
        "title": "search_title",
        "titleUrl": "url",
    }
)

core_columns = [
    "Participant ID",
    "time",
    "header",
    "search_title",
    "url",
    "details",
    "description",
]

search_histories = search_histories.reindex(columns=core_columns)
search_histories.head()
```

Parse Timestamps

Takeout timestamps are ISO-formatted UTC strings. Some rows include milliseconds and some do not, so we use pandas' ISO8601 parser explicitly.

```
search_histories["time"] = pd.to_datetime(
    search_histories["time"],
    format="ISO8601",
    utc=True,
)

search_histories["time"].agg(["min", "max"])
```

Decode Search Queries From URLs

YouTube search-history rows point to search-results URLs such as https://www.youtube.com/results?search_query=demo+search+query. Decoding this URL gives a language-neutral query field that later notebooks can use for temporal matching.

```
def extract_query_from_url(url):
    if not isinstance(url, str):
        return pd.NA
    parsed = urlparse(url)
    is_youtube_results_url = (
        parsed.scheme == "https"
        and parsed.netloc == "www.youtube.com"
        and parsed.path == "/results"
    )
    if not is_youtube_results_url:
        return pd.NA
    query_values = parse_qs(parsed.query, keep_blank_values=False).get("search_query", [])
    if not query_values or not query_values[0].strip():
        return pd.NA
    return query_values[0].strip()

search_histories["query_from_url"] = search_histories["url"].apply(extract_query_from_url)
search_histories["has_standard_search_url"] = search_histories["query_from_url"].notna()

search_histories[["search_title", "url", "query_from_url"]].head()
```

Identify Multilingual Search Actions

Google localizes the action phrase in Takeout titles according to the donor account language. The original workflow identified genuine search rows by extracting an action phrase and retaining known search actions such as [Searched](#), [Søgte](#), [Hai cercato](#), [Szukano](#), and [Выполнен поиск](#).

We keep that logic explicit here and add transparent columns so the classification can be audited.

```
SEARCH_ACTIONS = pd.DataFrame(
    [
        {"search_action": "Søgte", "language_hint": "Danish/Norwegian"},
        {"search_action": "Searched", "language_hint": "English"},
        {"search_action": "Hai cercato", "language_hint": "Italian"},
        {"search_action": "Szukano", "language_hint": "Polish"},
        {"search_action": "Выполнен поиск", "language_hint": "Russian"},
    ]
)

known_search_actions = set(SEARCH_ACTIONS["search_action"])

SEARCH_ACTIONS
```

```

def infer_search_action(title):
    if not isinstance(title, str) or not title.strip():
        return pd.NA
    tokens = title.split()
    first_two = " ".join(tokens[:2])
    if first_two in known_search_actions:
        return first_two
    return tokens[0]

def query_from_title(title, search_action):
    if not isinstance(title, str) or search_action not in known_search_actions:
        return pd.NA
    remainder = title[len(search_action) :].strip()
    # English Takeout titles commonly say "Searched for ...".
    if search_action == "Searched":
        remainder = re.sub(r"^for\s+", "", remainder, flags=re.IGNORECASE).strip()
    return remainder if remainder else pd.NA

search_histories["search_action"] = search_histories["search_title"].apply(infer_search_action)
search_histories["is_known_search_action"] = search_histories["search_action"].isin(
    known_search_actions
)
search_histories["is_search_query"] = (
    search_histories["is_known_search_action"]
    & search_histories["has_standard_search_url"]
)
search_histories["query_from_title"] = search_histories.apply(
    lambda row: query_from_title(row["search_title"], row["search_action"]),
    axis=1,
)

search_histories[
    ["search_title", "search_action", "query_from_title", "query_from_url", "is_search_query"]
].head()

```

Diagnostics

These checks document what happened during cleaning. For the mock dataset, we expect 120 combined search-history rows and 90 retained search-query rows.

```

search_query_histories = (
    search_histories.loc[search_histories["is_search_query"]]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)

diagnostics = {
    "donor_files_loaded": len(search_files),
    "combined_search_history_rows": len(search_histories),
    "search_query_rows": len(search_query_histories),
    "standard_search_url_rows": int(search_histories["has_standard_search_url"].sum()),
    "known_search_action_rows": int(search_histories["is_known_search_action"].sum()),
    "unrecognized_action_rows": int((~search_histories["is_known_search_action"]).sum()),
    "time_min": search_histories["time"].min(),
    "time_max": search_histories["time"].max(),
}

pd.Series(diagnostics)

```

```

pd.DataFrame(
    {
        "all_search_rows": search_histories.groupby("Participant ID").size(),
        "search_query_rows": search_query_histories.groupby("Participant ID").size(),
    }
).fillna(0).astype(int)

```

```

action_diagnostics = (
    search_histories.assign(
        search_action=search_histories["search_action"].fillna("UNRECOGNIZED")
    )
    .groupby(["search_action", "is_known_search_action"], dropna=False)
    .size()
    .reset_index(name="rows")
    .sort_values("rows", ascending=False)
    .reset_index(drop=True)
)

action_diagnostics

```

Save Outputs

`search_histories.csv` keeps all standardized rows from the search-history exports.

`search_query_histories.csv` keeps the rows that this notebook can confidently treat as genuine YouTube search queries.

```

search_export_columns = [
    "Participant ID",
    "time",
    "header",
    "search_title",
    "search_action",
    "is_known_search_action",
    "is_search_query",
    "query_from_title",
    "query_from_url",
    "url",
    "details",
    "description",
]
search_query_columns = [
    "Participant ID",
    "time",
    "search_title",
    "search_action",
    "query_from_title",
    "query_from_url",
    "url",
]

search_histories_export = (
    search_histories.loc[:, search_export_columns]
    .sort_values(["Participant ID", "time"], ascending=[True, False])
    .reset_index(drop=True)
)
search_query_histories = search_query_histories.loc[:, search_query_columns]

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
search_histories_export.to_csv(SEARCH_HISTORIES_PATH, index=False)
search_query_histories.to_csv(SEARCH_QUERY_HISTORIES_PATH, index=False)

saved_outputs = pd.DataFrame(
    {
        "table": ["search_histories", "search_query_histories"],
        "path": [SEARCH_HISTORIES_PATH, SEARCH_QUERY_HISTORIES_PATH],
        "rows": [len(search_histories_export), len(search_query_histories)],
    }
)
saved_outputs["path"] = saved_outputs["path"].apply(
    lambda output_path: output_path.relative_to(PROJECT_ROOT).as_posix()
)

saved_outputs

```

Result

The workflow now has a combined donated search-history table and a narrower search-query table. The search-query table can be used later to estimate whether watch events occurred shortly after explicit YouTube searches.

Stored Table Previews

The cells below show the first five rows of each dataframe saved as a CSV.

```
stored_search_tables = {
    "search_histories.csv": search_histories_export,
    "search_query_histories.csv": search_query_histories,
}

for file_name, dataframe in stored_search_tables.items():
    print(f"{file_name} ({len(dataframe)} rows)")
    display(dataframe.head(5))
```

03 Scrape YouTube Metadata With yt-dlp

This notebook demonstrates how to build a video-level metadata table for the videos observed in donated YouTube watch histories.

It reads unique `video_id` values from `outputs/tables/video_histories.csv`, uses `yt_dlp` to request public YouTube metadata, and writes one metadata row per unique video to `outputs/tables/meta_data.csv`. **Public-data note.** The video IDs included in this public repository are scrambled fake IDs. They preserve the shape of the workflow but cannot be scraped from YouTube. To run this notebook for a real project, replace the input `video_id` values with authorized real YouTube IDs and replace `YOUR_REGISTERED_TOKEN` with your registered identifying token.

What This Notebook Produces

The main output is `outputs/tables/meta_data.csv`.

Each row represents one unique YouTube video ID. Metadata collection is attempted for every unique `video_id` in `video_histories.csv`. When `yt_dlp` cannot retrieve metadata, the row is still kept and the raw error message is stored in `error`.

Dependency Check

This notebook requires `yt_dlp`. The import check below does not install anything automatically; it prints the command to run if the package is missing.

```
from pathlib import Path
from importlib.util import find_spec
import json
import time

import pandas as pd

if find_spec("yt_dlp") is None:
    print('yt_dlp is not installed in this kernel.')
    print('Install it with: %pip install yt-dlp')
else:
    import yt_dlp as yt
    print(f"yt_dlp available: {yt.version.__version__}")
```

Locate The Project And Configure The Scraper

Replace `YOUR_REGISTERED_TOKEN` with the registered identifying token or User-Agent string agreed for your project. The original scraper used this value in the HTTP `User-Agent` header; this notebook does the same.

```
def find_project_root(start=None):
    start = Path.cwd() if start is None else Path(start)
    for candidate in [start, *start.parents]:
        has_quarto_project = (candidate / "_quarto.yml").exists()
        has_video_histories = (candidate / "outputs" / "tables" / "video_histories.csv").exists()
        if has_quarto_project and has_video_histories:
            return candidate
    raise FileNotFoundError(
        "Could not find the Quarto project root. Run this notebook from "
        "inside youtube_donation_dsa_method after creating video_histories.csv."
    )

PROJECT_ROOT = find_project_root()
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "tables"
VIDEO_HISTORIES_PATH = OUTPUT_DIR / "video_histories.csv"
META_DATA_PATH = OUTPUT_DIR / "meta_data.csv"
SUBTITLES_DIR = PROJECT_ROOT / "outputs" / "subtitles_tmp"

IDENTIFYING_USER_AGENT = "YOUR_REGISTERED_TOKEN"
SLEEP_SECONDS_BETWEEN_REQUESTS = 0.0
MAX_VIDEOS_FOR_TEST = None
SUBTITLE_LANGUAGES = ["en"]

print(f"Project folder: {PROJECT_ROOT.name}")
print(f"Input table: {VIDEO_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Output table: {META_DATA_PATH.relative_to(PROJECT_ROOT).as_posix()}")
```

Load Unique Video IDs

The metadata table is video-level, not watch-event-level. We therefore deduplicate the `video_id` values from `video_histories.csv` before scraping.

```
video_histories = pd.read_csv(VIDEO_HISTORIES_PATH, dtype={"video_id": "string"})

if "video_id" not in video_histories.columns:
    raise ValueError("video_histories.csv must contain a video_id column.")

unique_video_ids = sorted(video_histories["video_id"].dropna().unique())

if MAX_VIDEOS_FOR_TEST is not None:
    unique_video_ids = unique_video_ids[:MAX_VIDEOS_FOR_TEST]

pd.Series(
    {
        "watch_rows": len(video_histories),
        "unique_video_ids_to_scrape": len(unique_video_ids),
        "max_videos_for_test": MAX_VIDEOS_FOR_TEST,
    }
)
```

Metadata Schema

The fields below are a compact CSV version of the original scraper schema. The `error` column is intentionally part of the schema because unavailable, private, removed, or age-restricted videos can be analytically informative.

```
CORE_METADATA_FIELDS = [
    "id",
    "title",
    "description",
    "channel_id",
    "channel",
    "uploader",
    "duration",
    "view_count",
    "like_count",
    "comment_count",
    "channel_follower_count",
    "upload_date",
    "timestamp",
    "webpage_url",
    "categories",
    "tags",
    "language",
    "resolution",
    "width",
    "height",
    "fps",
    "aspect_ratio",
]

SUBTITLE_FIELDS = [
    "requested_subtitles",
    "subtitles",
    "available_manual_subtitle_languages",
    "available_auto_caption_languages",
    "downloaded_subtitle_languages",
    "downloaded_subtitle_file_count",
]

METADATA_FIELDS = CORE_METADATA_FIELDS + SUBTITLE_FIELDS + ["metadata_status", "error"]

METADATA_FIELDS
```

Scraper Helpers

The original code requested both manually provided subtitles and automatic captions in English, wrote VTT files temporarily, read them back into the metadata row, and deleted the temporary files. This notebook follows that logic while also recording which subtitle languages were available and downloaded.

```

class DummyLogger:
    def debug(self, msg):
        pass

    def warning(self, msg):
        pass

    def error(self, msg):
        pass

def validate_scraper_config():
    if IDENTIFYING_USER_AGENT == "YOUR_REGISTERED_TOKEN":
        raise ValueError(
            'Replace IDENTIFYING_USER_AGENT = "YOUR_REGISTERED_TOKEN" with your registered token before scraping.'
        )
    if find_spec("yt_dlp") is None:
        raise ImportError(
            'yt_dlp is not installed. Run: %pip install "yt-dlp[default,curl-cffi]"'
        )

def as_json_text(value):
    if value is None:
        return None
    if isinstance(value, (list, dict)):
        return json.dumps(value, ensure_ascii=False, default=str)
    return value

def comma_join_keys(value):
    if isinstance(value, dict):
        return ",".join(sorted(value.keys()))
    return None

def empty_metadata_row(video_id):
    row = {field: None for field in METADATA_FIELDS}
    row["id"] = video_id
    row["metadata_status"] = "pending"
    row["downloaded_subtitle_file_count"] = 0
    return row

```

```

def get_metadata(video_id, quiet=True, no_warnings=True):
    import yt_dlp as yt

    url = f"https://www.youtube.com/watch?v={video_id}"
    ydl_opts = {
        "quiet": quiet,
        "no_warnings": no_warnings,
        "logger": DummyLogger(),
        "skip_download": True,
        "writesubtitles": True,
        "writeautomaticsub": True,
        "subtitlesformat": "vtt",
        "subtitleslangs": SUBTITLE_LANGUAGES,
        "outtmpl": str(SUBTITLES_DIR / video_id),
        "http_headers": {"User-Agent": IDENTIFYING_USER_AGENT},
    }

    row = empty_metadata_row(video_id)

    try:
        with yt.YoutubeDL(ydl_opts) as ydl:
            # download=True allows subtitle files to be written; skip_download=True prevents video download.
            info = ydl.extract_info(url, download=True)

            for field in CORE_METADATA_FIELDS:
                row[field] = as_json_text(info.get(field))

            manual_subtitles = info.get("subtitles") or {}
            automatic_captions = info.get("automatic_captions") or {}
            requested_subtitles = info.get("requested_subtitles") or {}

            row["available_manual_subtitle_languages"] = comma_join_keys(manual_subtitles)
            row["available_auto_caption_languages"] = comma_join_keys(automatic_captions)
            row["requested_subtitles"] = as_json_text(requested_subtitles)

            subtitle_texts = []
            downloaded_languages = []
            for language, subtitle_info in requested_subtitles.items():
                subtitle_file = subtitle_info.get("filepath") if isinstance(subtitle_info, dict) else None
                if subtitle_file and Path(subtitle_file).exists():
                    subtitle_text = Path(subtitle_file).read_text(encoding="utf-8")
                    subtitle_texts.append(f"WEBVTT_LANGUAGE: {language}\n{subtitle_text}")
                    downloaded_languages.append(language)
            try:
                Path(subtitle_file).unlink()
            except OSError as exc:
                print(f"Warning: failed to delete subtitle file {subtitle_file}: {exc}")

            row["subtitles"] = "\n\n".join(subtitle_texts) if subtitle_texts else None
            row["downloaded_subtitle_languages"] = ",".join(downloaded_languages) if downloaded_languages else None
            row["downloaded_subtitle_file_count"] = len(downloaded_languages)
            row["metadata_status"] = "ok"

    except Exception as exc:
        # Keep the raw yt_dlp message. These messages often explain why metadata was unavailable.
        row["error"] = str(exc)
        row["metadata_status"] = "error"

    return row

```

Run The Scrape

This cell performs live requests. It will stop until `IDENTIFYING_USER_AGENT` is changed from `YOUR_REGISTERED_TOKEN`. The default delay between requests is `0` because this notebook assumes the project has permission to scrape; increase `SLEEP_SECONDS_BETWEEN_REQUESTS` if your access conditions require slower pacing.

```

validate_scraper_config()
SUBTITLES_DIR.mkdir(parents=True, exist_ok=True)

metadata_rows = []
total_ids = len(unique_video_ids)

for index, video_id in enumerate(unique_video_ids, start=1):
    print(f"Scraping {index}/{total_ids}: {video_id}")
    metadata_rows.append(get_metadata(video_id))

    if SLEEP_SECONDS_BETWEEN_REQUESTS and index < total_ids:
        time.sleep(SLEEP_SECONDS_BETWEEN_REQUESTS)

meta_data = pd.DataFrame(metadata_rows, columns=METADATA_FIELDS)
meta_data.to_csv(META_DATA_PATH, index=False)

META_DATA_PATH.relative_to(PROJECT_ROOT).as_posix()

```

Diagnostics

After scraping, these diagnostics summarize coverage and errors. Error rows are not failures of the notebook; they are data about videos that could not be retrieved or whose subtitles could not be downloaded.

```

diagnostics = {
    "unique_video_ids_requested": len(unique_video_ids),
    "metadata_rows_written": len(meta_data),
    "unique_metadata_ids": meta_data["id"].nunique(),
    "rows_with_errors": int(meta_data["error"].notna().sum()),
    "rows_with_subtitles": int(meta_data["subtitles"].notna().sum()),
}

pd.Series(diagnostics)

```

```

error_summary = (
    meta_data.loc[meta_data["error"].notna(), "error"]
    .value_counts()
    .rename_axis("error")
    .reset_index(name="rows")
)

error_summary.head(10)

```

Result

The stored metadata table is keyed by `id`, which is the YouTube `video_id`. It can now be joined back to `video_histories.csv` in the next notebook.

```

print(f"meta_data.csv ({len(meta_data)} rows)")
display(meta_data.head(5))

```

04 Link Watch Data To Metadata, Exposure Context, And Problematic-View Labels

This notebook links cleaned donated YouTube watch data to scraped YouTube metadata and adds event-level context markers: whether a watch closely followed a YouTube search, whether the watched video came from a channel the participant had subscribed to, whether the video is classified as a Short or Long, whether the watch occurred during the configured daytime or nighttime window, and whether a metadata error has been manually categorized as problematic.

The output is an event-level table: each row is still one watch event from the donated history, but it now carries video-level metadata, raw scrape errors, strict search-associated and subscribed-channel flags, local-time day/night labels, Shorts/Longs labels, and transparent problematic-view labels. This creates a shared data foundation for later notebooks on ratios and unavailable or problematic views.

The notebook uses the cleaned files created by the previous notebooks:

- `outputs/tables/video_histories.csv`
- `outputs/tables/meta_data.csv`
- `outputs/tables/search_query_histories.csv`
- `data/mock_takeout/*/YouTube and YouTube Music/subscriptions/subscriptions.csv`
- `data/manual/unavailable_content_classification_filled.csv`

It writes:

- `outputs/tables/video_histories_enriched.csv`
- `outputs/tables/metadata_linkage_report.csv`
- `outputs/tables/subscription_histories.csv`
- `outputs/tables/exposure_linkage_report.csv`
- `outputs/tables/error_types_by_video.csv`

```
from pathlib import Path
import re

import pandas as pd

pd.set_option("display.max_columns", 120)
pd.set_option("display.max_colwidth", 120)
```

Locate Project Files

The notebook is designed to run from the project root or from inside the `scripts` folder. Paths are kept relative to the Quarto project so rendered notebooks do not expose private local folders.

```

def find_project_root(start=None):
    start = Path.cwd() if start is None else Path(start)
    for candidate in [start, *start.parents]:
        has_quarto_project = (candidate / "_quarto.yml").exists()
        has_output_folder = (candidate / "outputs" / "tables").exists()
        if has_quarto_project and has_output_folder:
            return candidate
    raise FileNotFoundError(
        "Could not find the Quarto project root. Run this notebook from "
        "inside youtube_donation_dsa_method."
    )

PROJECT_ROOT = find_project_root()
MOCK_TAKEOUT_DIR = PROJECT_ROOT / "data" / "mock_takeout"
MANUAL_DIR = PROJECT_ROOT / "data" / "manual"
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "tables"

VIDEO_HISTORIES_PATH = OUTPUT_DIR / "video_histories.csv"
META_DATA_PATH = OUTPUT_DIR / "meta_data.csv"
PUBLIC_DEMO_META_DATA_PATH = PROJECT_ROOT / "data" / "demo_metadata" / "meta_data_public.csv"
ACTIVE_META_DATA_PATH = META_DATA_PATH if META_DATA_PATH.exists() else PUBLIC_DEMO_META_DATA_PATH
SEARCH_QUERY_HISTORIES_PATH = OUTPUT_DIR / "search_query_histories.csv"
MANUAL_CLASSIFICATION_PATH = MANUAL_DIR / "unavailable_content_classification_filled.csv"
ENRICHED_VIDEO_HISTORIES_PATH = OUTPUT_DIR / "video_histories_enriched.csv"
METADATA_LINKAGE_REPORT_PATH = OUTPUT_DIR / "metadata_linkage_report.csv"
SUBSCRIPTION_HISTORIES_PATH = OUTPUT_DIR / "subscription_histories.csv"
EXPOSURE_LINKAGE_REPORT_PATH = OUTPUT_DIR / "exposure_linkage_report.csv"
ERROR_TYPES_BY_VIDEO_PATH = OUTPUT_DIR / "error_types_by_video.csv"

print(f"Project folder: {PROJECT_ROOT.name}")
print(f"Watch input: {VIDEO_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Metadata input: {ACTIVE_META_DATA_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Search input: {SEARCH_QUERY_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Manual classification input: {MANUAL_CLASSIFICATION_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print("Subscription input: data/mock_takeout/*/YouTube and YouTube Music/subscriptions/subscriptions.csv")
print(f"Output folder: {OUTPUT_DIR.relative_to(PROJECT_ROOT).as_posix()}")

```

Configure Event Flags

These settings control event-level labels. The mock data represents Danish users, so the default timezone is [Europe/Copenhagen](#). The original day/night distinction treated [06:00–23:59](#) as day and [00:00–05:59](#) as night; the boundaries are configurable here.

```

ANALYSIS_TIMEZONE = "Europe/Copenhagen"
DAYTIME_START_HOUR = 6
DAYTIME_END_HOUR = 24
SHORT_MAX_DURATION_SECONDS = 180
SHORT_MAX_ASPECT_RATIO = 1

def validate_hour_boundary(value, label, allow_24=False):
    upper = 24 if allow_24 else 23
    if not isinstance(value, int) or value < 0 or value > upper:
        raise ValueError(f"{label} must be an integer from 0 to {upper}.")

validate_hour_boundary(DAYTIME_START_HOUR, "DAYTIME_START_HOUR")
validate_hour_boundary(DAYTIME_END_HOUR, "DAYTIME_END_HOUR", allow_24=True)
if DAYTIME_START_HOUR == DAYTIME_END_HOUR:
    raise ValueError("DAYTIME_START_HOUR and DAYTIME_END_HOUR must define a non-empty interval.")

def hour_is_in_interval(hours, start_hour, end_hour):
    if start_hour < end_hour:
        return hours.ge(start_hour) & hours.lt(end_hour)
    return hours.ge(start_hour) | hours.lt(end_hour)

configuration = pd.DataFrame(
    {
        "setting": [
            "ANALYSIS_TIMEZONE",
            "DAYTIME_START_HOUR",
            "DAYTIME_END_HOUR",
            "SHORT_MAX_DURATION_SECONDS",
            "SHORT_MAX_ASPECT_RATIO",
        ],
        "value": [
            ANALYSIS_TIMEZONE,
            DAYTIME_START_HOUR,
            DAYTIME_END_HOUR,
            SHORT_MAX_DURATION_SECONDS,
            SHORT_MAX_ASPECT_RATIO,
        ],
    }
)

configuration

```

Configure Problematic-View Classification

The raw scraper error is kept unchanged, but the original workflow also grouped similar error messages before manual classification. These settings make that process visible and editable: the regular expressions normalize common `yt_dlp` messages, the alias map handles known wording variants, and the manual CSV assigns each generalized message to a category and criticality code.

```

ACCOUNT_TERMINATED_MESSAGE = (
    "Video unavailable. This video is no longer available because the YouTube account "
    "associated with this video has been terminated."
)
ACCOUNT_TERMINATED_IS_PROBLEMATIC = True

ASSESSMENT_MAP = {
    "Y": "Problematic",
    "N": "Not problematic",
    "Platform external legal action": "Removed due to external legal rights",
}

# The current mock scrape sometimes produces this shorter wording. We map it to
# the existing manual category rather than inventing a new classification row.
ERROR_CLASSIFICATION_MESSAGE_ALIASES = {
    "This video is not available": "Video unavailable. This video is not available",
}

ANSI_ESCAPE = re.compile(r"\x1B\[([0-?]*[ -/]*[@-~])")
COUNTRY_PREFIX = "The uploader has not made this video available in your country"
ERROR_NORMALIZATION_RULES = [
    (r"(copyright claim)\s+by.*", r"\1", re.IGNORECASE),
    (
        r"Video unavailable\. This video contains content from .*?,.*?blocked it(?: in your country)? on copyright grounds",
        "Video unavailable. This video contains content from copyright owner(s) who have blocked it on copyright grounds",
        re.IGNORECASE,
    ),
    (r"Parse error.*", "Some sort of parse error", re.IGNORECASE),
    (
        r"This video is available to this channel's members on level: .*? \((or any higher level\)".*,
        "Members-only video",
        re.IGNORECASE,
    ),
    (r"^\.*?\[youtube\]\s+.*?:\s*", "", 0),
    (
        r"Video unavailable\. This video contains content from .*?\. It is not available in your country\.",
        "Video unavailable in your country.",
        re.IGNORECASE,
    ),
    (
        r"ERROR: Unable to download video subtitles for '.*?': ERROR:.*",
        "Unable to download subtitles",
        re.IGNORECASE,
    ),
    (r"Sign in to confirm your age\..*", "Age-restricted video. Sign-in required", re.IGNORECASE),
    (
        r"(Video unavailable\. This video contains content from )\[^\.]+\(\. It is not available\.\?)",
        r"\1creator\2",
        re.IGNORECASE,
    ),
]

def normalize_error_message(raw_error, video_id):
    if pd.isna(raw_error):
        return raw_error

    text = str(raw_error)
    if pd.notna(video_id):
        text = text.replace(str(video_id), "video_id")

    text = ANSI_ESCAPE.sub("", text)
    text = text.replace("ERROR: [youtube] video_id:", "")
    text = text.strip()

    for pattern, replacement, flags in ERROR_NORMALIZATION_RULES:
        text = re.sub(pattern, replacement, text, flags=flags).strip()

    if COUNTRY_PREFIX in text:
        text = COUNTRY_PREFIX

    return text.strip()

problematic_view_configuration = pd.DataFrame(
    {
        "setting": [
            "MANUAL_CLASSIFICATION_PATH",
            "ACCOUNT_TERMINATED_IS_PROBLEMATIC",
            "ERROR_NORMALIZATION_RULE_COUNT",
            "ERROR_CLASSIFICATION_ALIAS_COUNT",
        ],
        "value": [

```

```

        MANUAL_CLASSIFICATION_PATH.relative_to(PROJECT_ROOT).as_posix(),
        ACCOUNT_TERMINATED_IS_PROBLEMATIC,
        len(ERROR_NORMALIZATION_RULES),
        len(ERROR_CLASSIFICATION_MESSAGE_ALIASES),
    ],
}
)

problematic_view_configuration

```

Load Watch Events And Metadata

Watch history and metadata have different units of analysis. The watch table is event-level: the same video may appear multiple times for one or more participants. The metadata table is video-level: each video ID should appear once after deduplication.

The enriched output keeps the watch-event unit because later analyses need participant IDs and timestamps.

```

for path, label in [
    (VIDEO_HISTORIES_PATH, "video_histories.csv"),
    (ACTIVE_META_DATA_PATH, "metadata table"),
    (SEARCH_QUERY_HISTORIES_PATH, "search_query_histories.csv"),
    (MANUAL_CLASSIFICATION_PATH, "unavailable_content_classification_filled.csv"),
]:
    if not path.exists():
        raise FileNotFoundError(
            f"Missing {label}: {path.relative_to(PROJECT_ROOT).as_posix()}"
        )

video_histories = pd.read_csv(
    VIDEO_HISTORIES_PATH,
    dtype={"Participant ID": "string", "video_id": "string", "channel_id": "string"},
    low_memory=False,
)

# Full descriptions and subtitle text can be very large. They stay in meta_data.csv,
# while this event-level handoff keeps only fields needed by the next analysis steps.
large_text_metadata_columns = {"description", "subtitles", "requested_subtitles"}
meta_data = pd.read_csv(
    ACTIVE_META_DATA_PATH,
    dtype={"id": "string", "channel_id": "string"},
    usecols=lambda column: column not in large_text_metadata_columns,
    low_memory=False,
)

input_summary = pd.DataFrame(
    {
        "dataset": ["video_histories", "meta_data"],
        "rows": [len(video_histories), len(meta_data)],
        "unique_video_ids": [
            video_histories["video_id"].nunique(dropna=True),
            meta_data["id"].nunique(dropna=True),
        ],
    }
)

input_summary

```

Validate Required Columns

Before joining, we check that the two upstream notebooks produced the expected fields. This makes the notebook fail early if a preceding cleaning or scraping step changed its schema.

```
def ensure_columns(dataframe, required_columns, label):
    missing_columns = sorted(set(required_columns) - set(dataframe.columns))
    if missing_columns:
        raise ValueError(
            f"Missing columns in {label}: {missing_columns}. "
            f"Found: {list(dataframe.columns)}"
        )

required_watch_columns = {
    "Participant ID",
    "time",
    "video_id",
    "watched_title",
    "clean_title",
    "url",
}

required_metadata_columns = {
    "id",
    "title",
    "duration",
    "channel_id",
    "channel",
    "aspect_ratio",
    "width",
    "height",
    "error",
    "metadata_status",
}

ensure_columns(video_histories, required_watch_columns, "video_histories.csv")
ensure_columns(meta_data, required_metadata_columns, "meta_data.csv")

print("Required columns are present.")
```

Prepare Watch Events

The watch table already contains one row per validated video watch. Here we parse time, clean the join key, and rename channel fields from the donation so they do not get confused with metadata channel fields.

```
video_histories = video_histories.copy()
watch_event_count = len(video_histories)
unique_watched_video_count = video_histories["video_id"].nunique(dropna=True)

video_histories["Participant ID"] = video_histories["Participant ID"].astype("string").str.strip()
video_histories["video_id"] = video_histories["video_id"].astype("string").str.strip()
video_histories["time"] = pd.to_datetime(
    video_histories["time"],
    utc=True,
    format="mixed",
    errors="coerce",
)

video_histories = video_histories.rename(
    columns={
        "channel_title": "donated_channel_title",
        "channel_url": "donated_channel_url",
        "channel_id": "donated_channel_id",
    }
)

if video_histories["time"].isna().any():
    raise ValueError("Some watch events have invalid timestamps.")

valid_video_id = video_histories["video_id"].str.match(r"^[A-Za-z0-9_-]{11}$", na=False)
if not valid_video_id.all():
    invalid_examples = video_histories.loc[~valid_video_id, "video_id"].head(10).tolist()
    raise ValueError(f"Some watch events have invalid video_id values: {invalid_examples}")

video_histories.head()
```

Prepare And Deduplicate Metadata

Scraping pipelines can produce duplicate metadata rows when several scrape batches are combined. Following the original metadata-combination logic, we keep one row per video ID and prefer rows with usable metadata: successful status, title, duration, and channel ID.

```
def has_text(series):
    return series.notna() & series.astype("string").str.strip().ne("").fillna(False)

meta_data = meta_data.copy()
metadata_rows_before_deduplication = len(meta_data)
metadata_duplicate_id_rows = int(meta_data.duplicated("id").sum())

meta_data["id"] = meta_data["id"].astype("string").str.strip()
meta_data["metadata_status"] = meta_data["metadata_status"].astype("string").str.strip().str.lower()

for numeric_column in [
    "duration",
    "view_count",
    "like_count",
    "comment_count",
    "channel_follower_count",
    "timestamp",
    "width",
    "height",
    "fps",
    "aspect_ratio",
]:
    if numeric_column in meta_data.columns:
        meta_data[numeric_column] = pd.to_numeric(meta_data[numeric_column], errors="coerce")

if "upload_date" in meta_data.columns:
    meta_data["upload_date"] = meta_data["upload_date"].astype("string").str.replace(r"\.0$", "", regex=True)

metadata_with_id = meta_data[has_text(meta_data["id"])].copy()
metadata_rows_without_id = len(meta_data) - len(metadata_with_id)

metadata_with_id["_status_priority"] = metadata_with_id["metadata_status"].eq("ok").fillna(False).astype(int)
metadata_with_id["_title_priority"] = has_text(metadata_with_id["title"]).astype(int)
metadata_with_id["_duration_priority"] = metadata_with_id["duration"].notna().astype(int)
metadata_with_id["_channel_priority"] = has_text(metadata_with_id["channel_id"]).astype(int)

metadata_deduplicated = (
    metadata_with_id
    .sort_values(
        [
            "id",
            "_status_priority",
            "_title_priority",
            "_duration_priority",
            "_channel_priority",
        ],
        ascending=[True, False, False, False, False],
        kind="mergesort",
    )
    .drop_duplicates(subset=["id"], keep="first")
    .drop(
        columns=[
            "_status_priority",
            "_title_priority",
            "_duration_priority",
            "_channel_priority",
        ]
    )
)

metadata_deduplicated.shape
```

Preserve Metadata Errors

The scraper stores raw `yt_dlp` error messages because they often contain valuable information: private video, age restriction, account termination, copyright block, country block, subtitle download failure, or other availability signals.

The raw message is kept unchanged as `metadata_error`. The next step creates separate normalized and manually classified fields, so readers can inspect both the original scrape message and the analytical category derived from it.

```
# Keep raw scrape errors unchanged. Missing errors stay missing.
metadata_deduplicated["error"] = metadata_deduplicated["error"].where(
    metadata_deduplicated["error"].notna(),
    pd.NA,
)

metadata_deduplicated[["id", "metadata_status", "error"]].head()
```

Classify Metadata Errors

The classification has two layers. First, raw scraper errors are normalized into generalized messages so that video IDs, creator names, and repeated `yt_dlp` prefixes do not create artificial categories. Second, the generalized message is matched to the manual classification table from the original workflow.

If a future scrape creates a generalized message that is not in the manual table or the explicit alias map, the notebook stops and prints the missing messages. This is intentional: new availability messages should be reviewed rather than silently assigned.

```

manual_map = pd.read_csv(MANUAL_CLASSIFICATION_PATH, dtype=str, low_memory=False)
manual_map = manual_map.rename(
    columns={
        "message": "generalized_error_msg",
        "classification": "category",
        "Critical": "critical",
    }
)
ensure_columns(
    manual_map,
    {"generalized_error_msg", "category", "critical"},
    "manual classification CSV",
)
manual_map = manual_map[["generalized_error_msg", "category", "critical"]].copy()
manual_map["generalized_error_msg"] = manual_map["generalized_error_msg"].astype(str).str.strip()
manual_map["category"] = manual_map["category"].fillna("").astype(str).str.strip()
manual_map["critical"] = manual_map["critical"].fillna("").astype(str).str.strip()

if manual_map["generalized_error_msg"].duplicated().any():
    duplicates = manual_map.loc[
        manual_map["generalized_error_msg"].duplicated(),
        "generalized_error_msg",
    ].unique().tolist()
    raise ValueError(f"Duplicate generalized errors in manual classification file: {duplicates[:10]}")

error_source = metadata_deduplicated[["id", "error"]].rename(
    columns={"id": "video_id", "error": "raw_error"}
).copy()
error_source["video_id"] = error_source["video_id"].astype("string").str.strip()
error_source["raw_error"] = error_source["raw_error"].where(has_text(error_source["raw_error"]), pd.NA)

errored_videos = error_source[has_text(error_source["raw_error"])].copy()
errored_videos["generalized_error_msg"] = errored_videos.apply(
    lambda row: normalize_error_message(row["raw_error"], row["video_id"]),
    axis=1,
)

errored_videos["classification_message"] = errored_videos["generalized_error_msg"].replace(
    ERROR_CLASSIFICATION_MESSAGE_ALIASES
)
alias_applied_count = int(
    errored_videos["classification_message"].ne(errored_videos["generalized_error_msg"]).sum()
)
errored_videos["generalized_error_msg"] = errored_videos["classification_message"]
errored_videos = errored_videos.drop(columns=["classification_message"])

missing_messages = sorted(
    set(errored_videos["generalized_error_msg"]) - set(manual_map["generalized_error_msg"])
)
if missing_messages:
    missing_error_messages = pd.DataFrame({"generalized_error_msg": missing_messages})
    display(missing_error_messages)
    raise ValueError(
        "The manual classification table is missing generalized error messages. "
        "Review the table above and add the messages before continuing."
    )

classified_errors = errored_videos.merge(
    manual_map,
    on="generalized_error_msg",
    how="left",
    validate="m:1",
)
classified_errors = classified_errors.drop_duplicates(subset=["video_id"]).copy()

error_types_by_video = error_source[["video_id"]].drop_duplicates().merge(
    classified_errors[
        ["video_id", "raw_error", "generalized_error_msg", "category", "critical"]
    ],
    on="video_id",
    how="left",
    validate="1:1",
)

error_types_by_video["category"] = error_types_by_video["category"].fillna("No error")
error_types_by_video["critical"] = error_types_by_video["critical"].fillna("").astype(str).str.strip()
error_types_by_video["generalized_error_msg"] = error_types_by_video["generalized_error_msg"].fillna("No error")
error_types_by_video["assessment_label"] = (
    error_types_by_video["critical"].map(ASSESSMENT_MAP).fillna("No error")
)
error_types_by_video["is_problematic"] = error_types_by_video["critical"].str.upper().eq("Y")

terminated_mask = error_types_by_video["generalized_error_msg"].eq(ACCOUNT_TERMINATED_MESSAGE)
terminated_critical = "Y" if ACCOUNT_TERMINATED_IS_PROBLEMATIC else "N"
terminated_label = "Problematic" if ACCOUNT_TERMINATED_IS_PROBLEMATIC else "Not problematic"

```

```

error_types_by_video.loc[terminated_mask, "critical"] = terminated_critical
error_types_by_video.loc[terminated_mask, "assessment_label"] = terminated_label
error_types_by_video.loc[terminated_mask, "is_problematic"] = ACCOUNT_TERMINATED_IS_PROBLEMATIC

error_types_by_video = error_types_by_video[
    [
        "video_id",
        "raw_error",
        "generalized_error_msg",
        "category",
        "critical",
        "assessment_label",
        "is_problematic",
    ]
].sort_values("video_id").reset_index(drop=True)

if error_types_by_video["video_id"].duplicated().any():
    raise AssertionError("Expected one row per video_id in error_types_by_video.")

error_classification_summary = pd.DataFrame(
    {
        "measure": [
            "metadata_videos",
            "metadata_videos_with_raw_error",
            "unique_generalized_error_messages",
            "alias_applied_videos",
            "unclassified_generalized_messages",
            "problematic_metadata_videos",
            "terminated_account_videos",
        ],
        "value": [
            len(error_types_by_video),
            int(error_types_by_video["raw_error"].notna().sum()),
            int(error_types_by_video.loc[error_types_by_video["raw_error"].notna(), "generalized_error_msg"].nunique()),
            alias_applied_count,
            len(missing_messages),
            int(error_types_by_video["is_problematic"].sum()),
            int(terminated_mask.sum()),
        ],
    }
)

error_classification_summary

```

Rename Metadata Fields

Several fields exist in both donation data and metadata. Prefixing metadata fields keeps the source of each column clear after the join.

```

metadata_rename_map = {
    "id": "video_id",
    "title": "metadata_title",
    "channel_id": "metadata_channel_id",
    "channel": "metadata_channel",
    "uploader": "metadata_uploader",
    "duration": "duration_seconds",
    "timestamp": "metadata_timestamp",
    "webpage_url": "metadata_url",
    "error": "metadata_error",
}

metadata_for_join = metadata_deduplicated.rename(columns=metadata_rename_map)

metadata_columns_to_keep = [
    "video_id",
    "metadata_title",
    "metadata_channel_id",
    "metadata_channel",
    "metadata_uploader",
    "duration_seconds",
    "view_count",
    "like_count",
    "comment_count",
    "channel_follower_count",
    "upload_date",
    "metadata_timestamp",
    "metadata_url",
    "categories",
    "tags",
    "language",
    "resolution",
    "width",
    "height",
    "fps",
    "aspect_ratio",
    "metadata_status",
    "metadata_error",
]

metadata_columns_to_keep = [
    column for column in metadata_columns_to_keep
    if column in metadata_for_join.columns
]

metadata_for_join = metadata_for_join[metadata_columns_to_keep].copy()

metadata_for_join.head()

```

Join Watch Events To Metadata

The join is a left join from watch history to metadata. That means no donated watch events are dropped. If metadata is missing or errored, the event remains in the output and the linkage columns make that visible.

```

def optional_column(dataframe, column):
    if column in dataframe.columns:
        return dataframe[column]
    return pd.Series(pd.NA, index=dataframe.index)

video_histories_enriched = video_histories.merge(
    metadata_for_join,
    on="video_id",
    how="left",
    validate="m:1",
    indicator=True,
)

video_histories_enriched["has_metadata"] = video_histories_enriched["_merge"].eq("both")
metadata_error_has_text = has_text(video_histories_enriched["metadata_error"])
metadata_status_is_error = (
    video_histories_enriched["metadata_status"]
    .astype("string")
    .str.lower()
    .eq("error")
    .fillna(False)
)

video_histories_enriched["metadata_join_status"] = "ok"
video_histories_enriched.loc[
    ~video_histories_enriched["has_metadata"],
    "metadata_join_status",
] = "missing_metadata"
video_histories_enriched.loc[
    video_histories_enriched["has_metadata"] & (metadata_error_has_text | metadata_status_is_error),
    "metadata_join_status",
] = "metadata_error"

video_histories_enriched["has_duration"] = video_histories_enriched["duration_seconds"].notna()

video_histories_enriched["analysis_channel_id"] = video_histories_enriched["metadata_channel_id"].combine_first(
    optional_column(video_histories_enriched, "donated_channel_id")
)
video_histories_enriched["analysis_channel_title"] = (
    video_histories_enriched["metadata_channel"]
    .combine_first(optional_column(video_histories_enriched, "metadata_uploader"))
    .combine_first(optional_column(video_histories_enriched, "donated_channel_title"))
)
video_histories_enriched["has_channel_id"] = has_text(video_histories_enriched["analysis_channel_id"])

video_histories_enriched = video_histories_enriched.drop(columns=["_merge"])

video_histories_enriched.shape

```

Add Problematic-View Labels

The video-level error catalog is joined back onto the event-level watch table. This keeps the unit of analysis as watch events while using one stable classification per unique video ID. The raw `metadata_error` column remains unchanged; the added columns show the normalized message and manual assessment.

```

problematic_labels_for_join = error_types_by_video.rename(
    columns={
        "category": "problematic_category",
        "critical": "problematic_critical",
        "assessment_label": "problematic_assessment_label",
    }
)[
    [
        "video_id",
        "generalized_error_msg",
        "problematic_category",
        "problematic_critical",
        "problematic_assessment_label",
        "is_problematic",
    ]
]

video_histories_enriched = video_histories_enriched.merge(
    problematic_labels_for_join,
    on="video_id",
    how="left",
    validate="m:1",
)

missing_problematic_lookup = video_histories_enriched["generalized_error_msg"].isna()
if missing_problematic_lookup.any():
    video_histories_enriched.loc[missing_problematic_lookup, "generalized_error_msg"] = "Missing metadata"
    video_histories_enriched.loc[missing_problematic_lookup, "problematic_category"] = "Missing metadata"
    video_histories_enriched.loc[missing_problematic_lookup, "problematic_critical"] = ""
    video_histories_enriched.loc[missing_problematic_lookup, "problematic_assessment_label"] = "Missing metadata"
    video_histories_enriched.loc[missing_problematic_lookup, "is_problematic"] = False

video_histories_enriched["is_problematic"] = video_histories_enriched["is_problematic"].astype("boolean")

metadata_error_event_mask = video_histories_enriched["metadata_join_status"].eq("metadata_error")
if video_histories_enriched.loc[metadata_error_event_mask, "problematic_category"].isna().any():
    raise AssertionError("Some metadata-error watch events are missing problematic-view categories.")

video_histories_enriched[
    [
        "video_id",
        "metadata_error",
        "generalized_error_msg",
        "problematic_category",
        "problematic_critical",
        "problematic_assessment_label",
        "is_problematic",
    ]
].head()

```

Add Shorts/Longs And Day/Night Flags

These are full-dataset event labels, not period summaries. The timestamp stays in UTC as `time`; local-time helper columns are added only so daytime and nighttime can be interpreted in the configured timezone.

Shorts are identified with the same metadata rule used in the original workflow: vertical or square-ish videos with duration at most 180 seconds. Missing Shorts evidence is treated as not short, matching the original downstream handling.

```

video_histories_enriched["watch_local_time"] = video_histories_enriched["time"].dt.tz_convert(ANALYSIS_TIMEZONE)
video_histories_enriched["watch_local_date"] = video_histories_enriched["watch_local_time"].dt.strftime("%Y-%m-%d")
video_histories_enriched["watch_local_hour"] = video_histories_enriched["watch_local_time"].dt.hour

is_daytime = hour_is_in_interval(
    video_histories_enriched["watch_local_hour"],
    DAYTIME_START_HOUR,
    DAYTIME_END_HOUR,
)
video_histories_enriched["is_daytime_watch"] = is_daytime.astype("boolean")
video_histories_enriched["is_nighttime_watch"] = (~is_daytime).astype("boolean")
video_histories_enriched["time_of_day"] = pd.Series("night", index=video_histories_enriched.index, dtype="string")
video_histories_enriched.loc[video_histories_enriched["is_daytime_watch"], "time_of_day"] = "day"

video_histories_enriched["shorts_metadata_available"] = (
    video_histories_enriched["aspect_ratio"].notna()
    & video_histories_enriched["duration_seconds"].notna()
)
video_histories_enriched["is_short"] = (
    video_histories_enriched["shorts_metadata_available"]
    & video_histories_enriched["aspect_ratio"].le(SHORT_MAX_ASPECT_RATIO)
    & video_histories_enriched["duration_seconds"].le(SHORT_MAX_DURATION_SECONDS)
).astype("boolean")
video_histories_enriched["is_long"] = (~video_histories_enriched["is_short"]).astype("boolean")

# Backward-compatible alias for readers who saw the earlier metadata-link notebook.
video_histories_enriched["is_short_by_metadata"] = video_histories_enriched["is_short"]

if not (video_histories_enriched["is_daytime_watch"] ^ video_histories_enriched["is_nighttime_watch"]).all():
    raise AssertionError("Each event should be exactly one of daytime or nighttime.")
if not (video_histories_enriched["is_short"] ^ video_histories_enriched["is_long"]).all():
    raise AssertionError("Each event should be exactly one of short or long.")

event_flag_diagnostics = pd.DataFrame(
    {
        "measure": [
            "analysis_timezone",
            "daytime_start_hour",
            "daytime_end_hour",
            "short_max_duration_seconds",
            "short_max_aspect_ratio",
            "shorts_watch_events",
            "longs_watch_events",
            "watch_events_missing_shorts_metadata",
            "daytime_watch_events",
            "nighttime_watch_events",
        ],
        "value": [
            ANALYSIS_TIMEZONE,
            DAYTIME_START_HOUR,
            DAYTIME_END_HOUR,
            SHORT_MAX_DURATION_SECONDS,
            SHORT_MAX_ASPECT_RATIO,
            int(video_histories_enriched["is_short"].sum()),
            int(video_histories_enriched["is_long"].sum()),
            int((~video_histories_enriched["shorts_metadata_available"]).sum()),
            int(video_histories_enriched["is_daytime_watch"].sum()),
            int(video_histories_enriched["is_nighttime_watch"].sum()),
        ],
    }
)

event_flag_diagnostics

```

Add Search-Associated Watch Marker

The original full-data workflow estimated a post-search interval from the distribution of search-watch distances. Here we do not refit that model on mock data. Instead, we apply the validated interval from the original analysis as a fixed methodological parameter: a watch is marked as searched when a participant search occurred 1 to 55 seconds before the watch event.

Only search timestamps and participant IDs are used. Search query text is intentionally not copied into the enriched watch table.

```

SEARCH_INTERVALS_SEC = [(1, 55)]

search_query_histories = pd.read_csv(
    SEARCH_QUERY_HISTORIES_PATH,
    usecols=["Participant ID", "time"],
    dtype={"Participant ID": "string"},
    low_memory=False,
)
ensure_columns(search_query_histories, {"Participant ID", "time"}, "search_query_histories.csv")

search_query_histories["Participant ID"] = search_query_histories["Participant ID"].astype("string").str.strip()
search_query_histories["time"] = pd.to_datetime(
    search_query_histories["time"],
    utc=True,
    format="mixed",
    errors="coerce",
)
search_query_histories = search_query_histories.dropna(subset=["Participant ID", "time"]).copy()
search_participant_ids = set(search_query_histories["Participant ID"].astype(str))

video_histories_enriched["has_search_history"] = (
    video_histories_enriched["Participant ID"].astype(str).isin(search_participant_ids)
)
video_histories_enriched["searched_for"] = pd.Series(pd.NA, index=video_histories_enriched.index, dtype="boolean")
video_histories_enriched.loc[video_histories_enriched["has_search_history"], "searched_for"] = False
video_histories_enriched["seconds_since_prior_search"] = pd.NA

watch_lookup = video_histories_enriched[["Participant ID", "time"]].copy()
watch_lookup["watch_event_position"] = video_histories_enriched.index

matched_search_parts = []
search_lookup = search_query_histories[["Participant ID", "time"]].rename(columns={"time": "search_time"})

for left_seconds, right_seconds in SEARCH_INTERVALS_SEC:
    if left_seconds < 0 or right_seconds < left_seconds:
        raise ValueError(f"Invalid search interval: {(left_seconds, right_seconds)}")

    interval_lookup = watch_lookup.copy()
    interval_lookup["search_lookup_time"] = interval_lookup["time"] - pd.to_timedelta(left_seconds, unit="s")

    participant_matches = []
    for participant_id, participant_watches in interval_lookup.groupby("Participant ID", sort=False):
        participant_searches = (
            search_lookup.loc[search_lookup["Participant ID"].eq(participant_id), ["search_time"]]
            .sort_values("search_time")
        )
        participant_watches = participant_watches.sort_values("search_lookup_time")

        if participant_searches.empty:
            participant_watches["search_time"] = pd.NaT
        else:
            participant_watches = pd.merge_asof(
                participant_watches,
                participant_searches,
                left_on="search_lookup_time",
                right_on="search_time",
                direction="backward",
                tolerance=pd.to_timedelta(right_seconds - left_seconds, unit="s"),
            )
        participant_matches.append(participant_watches)

    interval_matches = pd.concat(participant_matches, ignore_index=True)
    interval_matches["seconds_since_prior_search"] = (
        interval_matches["time"] - interval_matches["search_time"]
    ).dt.total_seconds()
    interval_matches = interval_matches[interval_matches["search_time"].notna()].copy()
    matched_search_parts.append(interval_matches)

if matched_search_parts:
    matched_searches = pd.concat(matched_search_parts, ignore_index=True)
    matched_searches = (
        matched_searches
        .sort_values(["watch_event_position", "seconds_since_prior_search"])
        .drop_duplicates(subset=["watch_event_position"], keep="first")
    )
    matched_positions = matched_searches["watch_event_position"].astype(int)
    video_histories_enriched.loc[matched_positions, "searched_for"] = True
    video_histories_enriched.loc[matched_positions, "seconds_since_prior_search"] = (
        matched_searches.set_index("watch_event_position")["seconds_since_prior_search"]
    )
else:
    matched_searches = pd.DataFrame(
        columns=["watch_event_position", "search_time", "seconds_since_prior_search"]
    )

```

```

search_diagnostics = pd.DataFrame(
    {
        "measure": [
            "search_query_rows",
            "participants_with_search_history",
            "search_intervals_applied",
            "searched_watch_events",
            "not_searched_watch_events",
            "watch_events_without_search_history",
        ],
        "value": [
            len(search_query_histories),
            len(search_participant_ids),
            str(SEARCH_INTERVALS_SEC),
            int(video_histories_enriched["searched_for"].eq(True).sum()),
            int(video_histories_enriched["searched_for"].eq(False).sum()),
            int(video_histories_enriched["searched_for"].isna().sum()),
        ],
    }
)

search_diagnostics

```

Add Subscribed-Channel Marker

The subscription marker follows the original method: compare each watched video's metadata-derived `metadata_channel_id` to the participant's donated subscription channel IDs.

The enriched table already contains `analysis_channel_id`, which can fall back to channel information from the donated watch history. We keep that field for later sensitivity checks, but the strict `subscribed` marker below uses only `metadata_channel_id` to match the original workflow.

```

subscription_files = sorted(
    MOCK_TAKEOUT_DIR.glob("*/YouTube and YouTube Music/subscriptions/subscriptions.csv")
)

subscription_frames = []
for path in subscription_files:
    participant_id = path.relative_to(MOCK_TAKEOUT_DIR).parts[0]
    subscriptions = pd.read_csv(path, dtype="string", low_memory=False)
    subscriptions["Participant ID"] = participant_id
    subscription_frames.append(subscriptions)

if subscription_frames:
    subscription_histories = pd.concat(subscription_frames, ignore_index=True)
else:
    subscription_histories = pd.DataFrame(
        columns=["Participant ID", "Channel Id", "Channel Url", "Channel Title"]
    )

subscription_histories = subscription_histories.rename(
    columns={
        "Channel Id": "subscribed_channel_id",
        "Channel Url": "subscribed_channel_url",
        "Channel Title": "subscribed_channel_title",
    }
)
ensure_columns(
    subscription_histories,
    {"Participant ID", "subscribed_channel_id", "subscribed_channel_url", "subscribed_channel_title",
     "subscription histories",
    }
)

subscription_histories = subscription_histories[
    ["Participant ID", "subscribed_channel_id", "subscribed_channel_url", "subscribed_channel_title"]
].copy()
subscription_histories["Participant ID"] = subscription_histories["Participant ID"].astype("string").str.strip()
subscription_histories["subscribed_channel_id"] =
subscription_histories["subscribed_channel_id"].astype("string").str.strip()
subscription_histories = subscription_histories.dropna(subset=["Participant ID",
"subscribed_channel_id"]).drop_duplicates()

subscription_participant_ids = set(subscription_histories["Participant ID"].astype(str))
subscribed_channel_pairs = set(
    zip(
        subscription_histories["Participant ID"].astype(str),
        subscription_histories["subscribed_channel_id"].astype(str),
    )
)

video_histories_enriched["has_subscription_history"] = (
    video_histories_enriched["Participant ID"].astype(str).isin(subscription_participant_ids)
)
video_histories_enriched["subscribed"] = pd.Series(pd.NA, index=video_histories_enriched.index, dtype="boolean")

has_metadata_channel_id = has_text(video_histories_enriched["metadata_channel_id"])
can_check_subscription = video_histories_enriched["has_subscription_history"] & has_metadata_channel_id

video_histories_enriched.loc[can_check_subscription, "subscribed"] = [
    (str(participant_id), str(channel_id)) in subscribed_channel_pairs
    for participant_id, channel_id in zip(
        video_histories_enriched.loc[can_check_subscription, "Participant ID"],
        video_histories_enriched.loc[can_check_subscription, "metadata_channel_id"],
    )
]

subscription_diagnostics = pd.DataFrame(
    {
        "measure": [
            "subscription_files",
            "subscription_rows",
            "participants_with_subscription_history",
            "subscribed_watch_events",
            "not_subscribed_watch_events",
            "watch_events_without_subscription_status",
            "watch_events_missing_metadata_channel_id",
        ],
        "value": [
            len(subscription_files),
            len(subscription_histories),
            len(subscription_participant_ids),
            int(video_histories_enriched["subscribed"].eq(True).sum()),
            int(video_histories_enriched["subscribed"].eq(False).sum()),
            int(video_histories_enriched["subscribed"].isna().sum()),
            int((~has_metadata_channel_id).sum()),
        ],
    }
)

```

```
)  
subscription_diagnostics
```

Select The Handoff Columns

The handoff table keeps the columns needed for the next notebooks, while avoiding very large metadata text fields. The original `meta_data.csv` remains the place to inspect descriptions and subtitle text, and `error_types_by_video.csv` stores the video-level error classification catalog.

```

preferred_output_columns = [
    "Participant ID",
    "time",
    "watch_local_time",
    "watch_local_date",
    "watch_local_hour",
    "time_of_day",
    "is_daytime_watch",
    "is_nighttime_watch",
    "video_id",
    "watched_title",
    "clean_title",
    "watch_action_prefix",
    "watch_action_language_hint",
    "url",
    "donated_channel_title",
    "donated_channel_url",
    "donated_channel_id",
    "metadata_title",
    "metadata_channel",
    "metadata_uploader",
    "metadata_channel_id",
    "analysis_channel_title",
    "analysis_channel_id",
    "duration_seconds",
    "aspect_ratio",
    "width",
    "height",
    "resolution",
    "fps",
    "shorts_metadata_available",
    "is_short",
    "is_long",
    "is_short_by_metadata",
    "view_count",
    "like_count",
    "comment_count",
    "channel_follower_count",
    "upload_date",
    "metadata_timestamp",
    "metadata_url",
    "categories",
    "tags",
    "language",
    "metadata_status",
    "metadata_join_status",
    "searched_for",
    "has_search_history",
    "seconds_since_prior_search",
    "subscribed",
    "has_subscription_history",
    "has_metadata",
    "has_duration",
    "has_channel_id",
    "metadata_error",
    "generalized_error_msg",
    "problematic_category",
    "problematic_critical",
    "problematic_assessment_label",
    "is_problematic",
]

output_columns = [
    column for column in preferred_output_columns
    if column in video_histories_enriched.columns
]

video_histories_enriched = video_histories_enriched[output_columns].copy()

video_histories_enriched.head()

```

Build Linkage Diagnostics

These checks document what the join did. They are useful both for teaching and for catching methodological problems before later notebooks compute ratios.

```

matched_events = video_histories_enriched["has_metadata"].sum()
missing_metadata_events = video_histories_enriched["metadata_join_status"].eq("missing_metadata").sum()
metadata_error_events = video_histories_enriched["metadata_join_status"].eq("metadata_error").sum()
missing_duration_events = (~video_histories_enriched["has_duration"]).sum()
missing_channel_id_events = (~video_histories_enriched["has_channel_id"]).sum()
shorts_candidate_events = video_histories_enriched["is_short"].eq(True).sum()

metadata_linkage_report = pd.DataFrame(
    [
        {"measure": "watch_events", "value": int(watch_event_count)},
        {"measure": "unique_watched_videos", "value": int(unique_watched_video_count)},
        {"measure": "metadata_rows_before_deduplication", "value": int(metadata_rows_before_deduplication)},
        {"measure": "metadata_rows_after_deduplication", "value": int(len(metadata_deduplicated))},
        {"measure": "metadata_duplicate_id_rows", "value": int(metadata_duplicate_id_rows)},
        {"measure": "metadata_rows_without_id", "value": int(metadata_rows_without_id)},
        {"measure": "matched_watch_events", "value": int(matched_events)},
        {"measure": "missing_metadata_watch_events", "value": int(missing_metadata_events)},
        {"measure": "metadata_error_watch_events", "value": int(metadata_error_events)},
        {"measure": "watch_events_missing_duration", "value": int(missing_duration_events)},
        {"measure": "watch_events_missing_channel_id", "value": int(missing_channel_id_events)},
        {"measure": "watch_events_flagged_short_by_metadata", "value": int(shorts_candidate_events)},
        {
            "measure": "matched_unique_videos",
            "value": int(video_histories_enriched.loc[video_histories_enriched["has_metadata"], "video_id"].nunique()),
        },
        {
            "measure": "missing_metadata_unique_videos",
            "value":
int(video_histories_enriched.loc[video_histories_enriched["metadata_join_status"].eq("missing_metadata"),
"video_id"].nunique()),
        },
        {
            "measure": "metadata_error_unique_videos",
            "value":
int(video_histories_enriched.loc[video_histories_enriched["metadata_join_status"].eq("metadata_error"),
"video_id"].nunique()),
        },
    ]
)

problematic_base_diagnostics = pd.DataFrame(
    [
        {
            "measure": "errored_unique_videos",
            "value": int(error_types_by_video.loc[error_types_by_video["raw_error"].notna(), "video_id"].nunique()),
        },
        {
            "measure": "unique_generalized_error_messages",
            "value": int(error_types_by_video.loc[error_types_by_video["raw_error"].notna(),
"generalized_error_msg"].nunique()),
        },
        {"measure": "alias_applied_videos", "value": int(alias_applied_count)},
        {"measure": "unclassified_generalized_messages", "value": int(len(missing_messages))},
        {"measure": "problematic_watch_events", "value": int(video_histories_enriched["is_problematic"].sum())},
        {
            "measure": "problematic_unique_videos",
            "value": int(video_histories_enriched.loc[video_histories_enriched["is_problematic"].eq(True),
"video_id"].nunique()),
        },
    ]
)

problematic_assessment_counts = pd.DataFrame(
    [
        {"measure": f"assessment_label_count::{label}", "value": int(count)}
        for label, count in video_histories_enriched["problematic_assessment_label"].value_counts(dropna=False).items()
    ]
)

problematic_category_counts = pd.DataFrame(
    [
        {"measure": f"category_count::{label}", "value": int(count)}
        for label, count in video_histories_enriched["problematic_category"].value_counts(dropna=False).items()
    ]
)

problematic_diagnostics = pd.concat(
    [problematic_base_diagnostics, problematic_assessment_counts, problematic_category_counts],
    ignore_index=True,
)

exposure_linkage_report = pd.concat(
    [
        search_diagnostics.assign(section="search"),
        subscription_diagnostics.assign(section="subscription"),
        event_flag_diagnostics.assign(section="event_flags"),
        problematic_diagnostics.assign(section="problematic_views"),
    ]
)

```

```
],
  ignore_index=True,
)[["section", "measure", "value"]]

metadata_linkage_report
```

Save Outputs

The enriched table is the main handoff. The linkage and exposure reports are compact audit trails, while `error_types_by_video.csv` keeps the video-level classification catalog behind the event-level labels.

```

if len(video_histories_enriched) != watch_event_count:
    raise AssertionError("The join changed the number of watch events.")

if video_histories_enriched["video_id"].isna().any():
    raise AssertionError("The enriched table contains missing video_id values.")

required_output_columns = {
    "duration_seconds",
    "aspect_ratio",
    "analysis_channel_id",
    "metadata_error",
    "searched_for",
    "has_search_history",
    "seconds_since_prior_search",
    "subscribed",
    "has_subscription_history",
    "watch_local_time",
    "watch_local_date",
    "watch_local_hour",
    "time_of_day",
    "is_daytime_watch",
    "is_nighttime_watch",
    "shorts_metadata_available",
    "is_short",
    "is_long",
    "generalized_error_msg",
    "problematic_category",
    "problematic_critical",
    "problematic_assessment_label",
    "is_problematic",
}

ensure_columns(video_histories_enriched, required_output_columns, "video_histories_enriched")
ensure_columns(
    error_types_by_video,
    {
        "video_id",
        "raw_error",
        "generalized_error_msg",
        "category",
        "critical",
        "assessment_label",
        "is_problematic",
    },
    "error_types_by_video",
)

unexpected_query_columns = sorted(
    set(video_histories_enriched.columns) & {"search_title", "query_from_title", "query_from_url"}
)

if unexpected_query_columns:
    raise AssertionError(f"Search query text leaked into the enriched table: {unexpected_query_columns}")

if error_types_by_video["video_id"].nunique(dropna=True) != len(error_types_by_video):
    raise AssertionError("error_types_by_video should contain one row per metadata video ID.")

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
video_histories_enriched.to_csv(ENRICHED_VIDEO_HISTORIES_PATH, index=False)
metadata_linkage_report.to_csv(METADATA_LINKAGE_REPORT_PATH, index=False)
subscription_histories.to_csv(SUBSCRIPTION_HISTORIES_PATH, index=False)
exposure_linkage_report.to_csv(EXPOSURE_LINKAGE_REPORT_PATH, index=False)
error_types_by_video.to_csv(ERROR_TYPES_BY_VIDEO_PATH, index=False)

for path in [
    ENRICHED_VIDEO_HISTORIES_PATH,
    METADATA_LINKAGE_REPORT_PATH,
    SUBSCRIPTION_HISTORIES_PATH,
    EXPOSURE_LINKAGE_REPORT_PATH,
    ERROR_TYPES_BY_VIDEO_PATH,
]:
    if not path.exists():
        raise AssertionError(f"Expected output was not created: {path.name}")

print(f"Saved {ENRICHED_VIDEO_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Saved {METADATA_LINKAGE_REPORT_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Saved {SUBSCRIPTION_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Saved {EXPOSURE_LINKAGE_REPORT_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Saved {ERROR_TYPES_BY_VIDEO_PATH.relative_to(PROJECT_ROOT).as_posix()}")

```

Result

The diagnostics summarize the metadata join, exposure markers, event flags, and problematic-view classification. The table previews show the first five rows of each CSV stored by this notebook.

```
print("Metadata diagnostics")
print(metadata_linkage_report.to_string(index=False))

print()
print("Exposure and classification diagnostics")
print(exposure_linkage_report.to_string(index=False))

print()
print("video_histories_enriched.csv")
display(video_histories_enriched.head(5))

print("metadata_linkage_report.csv")
display(metadata_linkage_report.head(5))

print("subscription_histories.csv")
display(subscription_histories.head(5))

print("exposure_linkage_report.csv")
display(exposure_linkage_report.head(5))

print("error_types_by_video.csv")
display(error_types_by_video.head(5))
```

05 Estimate Watch Time From Enriched Watch Events

This notebook estimates watch time from the enriched event table created in notebook 04.

The estimate follows the core logic of the original watch-time script: combine each video's metadata duration with the start time of the next observed watch event for the same participant. If the next watch starts before the current video's metadata duration has elapsed, the unelapsed duration is treated as skipped. If the next watch starts after the video's expected end, the full video duration is counted.

This is still an estimate. YouTube Takeout watch history gives start times, not completion times. The calculation is most useful for aggregate summaries across many events, and the assumptions should be reviewed before using it for substantive claims.

What This Notebook Produces

The notebook reads `outputs/tables/video_histories_enriched.csv` and writes three files:

- `outputs/tables/video_histories_watchtime.csv`: one row per watch event, with watch-time helper columns added.
- `outputs/tables/watchtime_summary.csv`: one row per participant with all-data watch-time totals by Shorts/Longs and day/night.
- `outputs/tables/watchtime_diagnostics.csv`: compact diagnostics and assumption settings.

Unlike the legacy script, this teaching version does not create separate one-month, six-month, or one-year windows. It runs across all events in the enriched mock dataset.

Locate Project Files

Paths are kept relative to the Quarto project root so the notebook can be rendered locally or as part of the website without exposing private machine paths.

```

from pathlib import Path

import pandas as pd

def find_project_root(start=None):
    start = Path.cwd() if start is None else Path(start)
    for candidate in [start, *start.parents]:
        has_quarto_project = (candidate / "_quarto.yml").exists()
        has_outputs = (candidate / "outputs" / "tables").exists()
        if has_quarto_project and has_outputs:
            return candidate
    raise FileNotFoundError(
        "Could not find the Quarto project root. Run this notebook from "
        "inside the public YouTube donation method repository."
    )

PROJECT_ROOT = find_project_root()
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "tables"

ENRICHED_VIDEO_HISTORIES_PATH = OUTPUT_DIR / "video_histories_enriched.csv"
WATCHTIME_EVENTS_PATH = OUTPUT_DIR / "video_histories_watchtime.csv"
WATCHTIME_SUMMARY_PATH = OUTPUT_DIR / "watchtime_summary.csv"
WATCHTIME_DIAGNOSTICS_PATH = OUTPUT_DIR / "watchtime_diagnostics.csv"

print(f"Project folder: {PROJECT_ROOT.name}")
print(f"Input table: {ENRICHED_VIDEO_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Output folder: {OUTPUT_DIR.relative_to(PROJECT_ROOT).as_posix()}")

```

Configure Assumptions

The key assumption is how to handle the last observed watch event for each participant. There is no next watch event that can reveal whether the participant stopped early. The conservative default is therefore to leave the last event's estimated watch time unknown.

Set `ASSUME_FULL_DURATION_FOR_LAST_EVENT = True` if a project wants to count the full metadata duration for each participant's last observed event. This is more complete but also stronger.

```

ASSUME_FULL_DURATION_FOR_LAST_EVENT = False
CAP_ESTIMATED_WATCHTIME_TO_DURATION = True

print("Assume full duration for last observed event:", ASSUME_FULL_DURATION_FOR_LAST_EVENT)
print("Cap estimated watch time to metadata duration:", CAP_ESTIMATED_WATCHTIME_TO_DURATION)

```

Load The Enriched Watch Table

Notebook 04 already joined watch events to metadata and created the day/night and Shorts/Longs labels. This notebook reuses those fields instead of recomputing timezone-sensitive classifications.

```

if not ENRICHED_VIDEO_HISTORIES_PATH.exists():
    raise FileNotFoundError(
        f'Missing input table: {ENRICHED_VIDEO_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}'
    )

video_histories_enriched = pd.read_csv(
    ENRICHED_VIDEO_HISTORIES_PATH,
    dtype={"Participant ID": "string", "video_id": "string"},
    low_memory=False,
)

required_columns = {
    "Participant ID",
    "time",
    "video_id",
    "duration_seconds",
    "is_short",
    "is_long",
    "time_of_day",
    "watch_local_time",
}
missing_columns = sorted(required_columns - set(video_histories_enriched.columns))
if missing_columns:
    raise ValueError(
        f'Missing required columns in video_histories_enriched.csv: {missing_columns}'
    )

video_histories_enriched["time"] = pd.to_datetime(
    video_histories_enriched["time"], utc=True, format="mixed", errors="coerce"
)
if video_histories_enriched["time"].isna().any():
    raise ValueError("Some watch events have invalid UTC timestamps.")

video_histories_enriched["duration_seconds"] = pd.to_numeric(
    video_histories_enriched["duration_seconds"], errors="coerce"
)

print(f"Loaded {len(video_histories_enriched)} enriched watch events")
print(f"Participants: {video_histories_enriched['Participant ID'].nunique()}")
print(f"Events with metadata duration: {video_histories_enriched['duration_seconds'].notna().sum()}")

video_histories_enriched.head(5)

```

Estimate Event-Level Watch Time

For each participant, the events are sorted by watch start time. The next watch start is then used as an upper bound for how long the current event could have been watched.

For rows with a known metadata duration and a known next watch start:

- if the next watch starts before the current video would have ended, estimated watch time is the time until the next watch;
- if the next watch starts after the current video would have ended, estimated watch time is the full metadata duration.

Rows without metadata duration remain unknown. Last observed events remain unknown by default, unless configured otherwise above.

```

video_histories_watchtime = video_histories_enriched.copy()
video_histories_watchtime["watch_event_position"] = range(len(video_histories_watchtime))

video_histories_watchtime = video_histories_watchtime.sort_values(
    ["Participant ID", "time", "watch_event_position"]
).reset_index(drop=True)

video_histories_watchtime["participant_event_number"] = (
    video_histories_watchtime.groupby("Participant ID").cumcount() + 1
)

video_histories_watchtime["next_watch_time"] = (
    video_histories_watchtime.groupby("Participant ID")["time"].shift(-1)
)

video_histories_watchtime["seconds_to_next_watch"] = (
    video_histories_watchtime["next_watch_time"] - video_histories_watchtime["time"]
).dt.total_seconds()

video_histories_watchtime["expected_end_time"] = (
    video_histories_watchtime["time"]
    + pd.to_timedelta(video_histories_watchtime["duration_seconds"], unit="s")
)

has_duration = video_histories_watchtime["duration_seconds"].notna()
has_next_watch = video_histories_watchtime["next_watch_time"].notna()
valid_duration = has_duration & video_histories_watchtime["duration_seconds"].ge(0)
valid_next_gap = has_next_watch & video_histories_watchtime["seconds_to_next_watch"].ge(0)

video_histories_watchtime["estimated_watch_seconds"] = pd.NA

bounded_rows = valid_duration & valid_next_gap
video_histories_watchtime.loc[bounded_rows, "estimated_watch_seconds"] = video_histories_watchtime.loc[
    bounded_rows, ["duration_seconds", "seconds_to_next_watch"]
].min(axis=1)

if ASSUME_FULL_DURATION_FOR_LAST_EVENT:
    last_rows_with_duration = valid_duration & ~has_next_watch
    video_histories_watchtime.loc[last_rows_with_duration, "estimated_watch_seconds"] = (
        video_histories_watchtime.loc[last_rows_with_duration, "duration_seconds"]
    )

video_histories_watchtime["estimated_watch_seconds"] = pd.to_numeric(
    video_histories_watchtime["estimated_watch_seconds"], errors="coerce"
)

if CAP_ESTIMATED_WATCHTIME_TO_DURATION:
    video_histories_watchtime["estimated_watch_seconds"] = video_histories_watchtime[
        ["estimated_watch_seconds", "duration_seconds"]
    ].min(axis=1)

video_histories_watchtime["estimated_watch_seconds"] = (
    video_histories_watchtime["estimated_watch_seconds"].clip(lower=0)
)

video_histories_watchtime["overlap_with_next_seconds"] = (
    video_histories_watchtime["duration_seconds"] - video_histories_watchtime["seconds_to_next_watch"]
).clip(lower=0)
video_histories_watchtime.loc[~bounded_rows, "overlap_with_next_seconds"] = pd.NA

video_histories_watchtime["estimated_watch_minutes"] = (
    video_histories_watchtime["estimated_watch_seconds"] / 60
)
video_histories_watchtime["estimated_watch_hours"] = (
    video_histories_watchtime["estimated_watch_seconds"] / 3600
)

video_histories_watchtime["watchtime_status"] = "estimated"
video_histories_watchtime.loc[~has_duration, "watchtime_status"] = "missing_duration"
video_histories_watchtime.loc[has_duration & video_histories_watchtime["duration_seconds"].lt(0), "watchtime_status"] =
"invalid_duration"
video_histories_watchtime.loc[valid_duration & has_next_watch & video_histories_watchtime["seconds_to_next_watch"].lt(0),
"watchtime_status"] = "invalid_next_watch_order"
video_histories_watchtime.loc[valid_duration & ~has_next_watch, "watchtime_status"] = (
    "assumed_full_duration_last_event" if ASSUME_FULL_DURATION_FOR_LAST_EVENT else "last_observed_event_unknown"
)
video_histories_watchtime.loc[
    bounded_rows & video_histories_watchtime["seconds_to_next_watch"].lt(video_histories_watchtime["duration_seconds"]),
    "watchtime_status",
] = "cut_off_by_next_watch"
video_histories_watchtime.loc[
    bounded_rows & video_histories_watchtime["seconds_to_next_watch"].ge(video_histories_watchtime["duration_seconds"]),
    "watchtime_status",
] = "full_duration_before_next_watch"

```

```
video_histories_watchtime[[
  "Participant ID",
  "time",
  "video_id",
  "duration_seconds",
  "next_watch_time",
  "seconds_to_next_watch",
  "estimated_watch_seconds",
  "watchtime_status",
]].head(10)
```

Build All-Data Participant Summaries

The legacy script exported one-month, six-month, and one-year summaries. This notebook keeps the same conceptual breakdown by participant, day/night, and Shorts/Longs, but runs it across the full mock dataset only.

The summary reports total estimated hours, not average daily hours, because no period window is imposed here.

```

summary_events = video_histories_watchtime.copy()
summary_events["video_format"] = "long"
summary_events.loc[summary_events["is_short"].astype("string").str.lower().eq("true"), "video_format"] = "short"
summary_events["has_estimated_watchtime"] = summary_events["estimated_watch_seconds"].notna()

watchtime_by_group = (
    summary_events
    .groupby(["Participant ID", "video_format", "time_of_day"], dropna=False)["estimated_watch_seconds"]
    .sum(min_count=1)
    .reset_index()
)
watchtime_by_group["summary_column"] = (
    "watchtime_"
    + watchtime_by_group["video_format"].astype(str)
    + "_"
    + watchtime_by_group["time_of_day"].astype(str)
    + "_hours"
)
watchtime_wide = (
    watchtime_by_group
    .pivot(index="Participant ID", columns="summary_column", values="estimated_watch_seconds")
    .reset_index()
)

for column in watchtime_wide.columns:
    if column != "Participant ID":
        watchtime_wide[column] = watchtime_wide[column] / 3600

base_summary = (
    summary_events
    .groupby("Participant ID")
    .agg(
        watch_events=("video_id", "size"),
        events_with_estimated_watchtime=("has_estimated_watchtime", "sum"),
        total_estimated_watch_seconds=("estimated_watch_seconds", "sum"),
        missing_duration_events=("watchtime_status", lambda s: (s == "missing_duration").sum()),
        last_observed_unknown_events=("watchtime_status", lambda s: (s == "last_observed_event_unknown").sum()),
        cut_off_by_next_watch_events=("watchtime_status", lambda s: (s == "cut_off_by_next_watch").sum()),
        full_duration_before_next_watch_events=("watchtime_status", lambda s: (s ==
"full_duration_before_next_watch").sum()),
    )
    .reset_index()
)
base_summary["total_estimated_watch_hours"] = base_summary["total_estimated_watch_seconds"] / 3600

watchtime_summary = base_summary.merge(watchtime_wide, on="Participant ID", how="left").fillna(0)

preferred_summary_columns = [
    "Participant ID",
    "watch_events",
    "events_with_estimated_watchtime",
    "missing_duration_events",
    "last_observed_unknown_events",
    "cut_off_by_next_watch_events",
    "full_duration_before_next_watch_events",
    "total_estimated_watch_seconds",
    "total_estimated_watch_hours",
    "watchtime_short_day_hours",
    "watchtime_short_night_hours",
    "watchtime_long_day_hours",
    "watchtime_long_night_hours",
]
for column in preferred_summary_columns:
    if column not in watchtime_summary.columns:
        watchtime_summary[column] = 0

watchtime_summary = watchtime_summary[preferred_summary_columns]

watchtime_summary

```

Diagnostics

These diagnostics make the estimation assumptions and missingness visible. In particular, missing duration and last observed events limit how much watch time can be estimated from the available data.

```

def count_status(status):
    return int((video_histories_watchtime["watchtime_status"] == status).sum())

watchtime_diagnostics = pd.DataFrame(
    [
        {"section": "inputs", "measure": "watch_events", "value": len(video_histories_watchtime)},
        {"section": "inputs", "measure": "participants", "value": video_histories_watchtime["Participant ID"].nunique()},
        {"section": "inputs", "measure": "events_with_duration", "value":
int(video_histories_watchtime["duration_seconds"].notna().sum())},
        {"section": "inputs", "measure": "events_missing_duration", "value":
int(video_histories_watchtime["duration_seconds"].isna().sum())},
        {"section": "sequence", "measure": "events_with_next_watch", "value":
int(video_histories_watchtime["next_watch_time"].notna().sum())},
        {"section": "sequence", "measure": "last_observed_events", "value":
int(video_histories_watchtime["next_watch_time"].isna().sum())},
        {"section": "watchtime", "measure": "events_with_estimated_watchtime", "value":
int(video_histories_watchtime["estimated_watch_seconds"].notna().sum())},
        {"section": "watchtime", "measure": "total_estimated_watch_hours", "value":
round(float(video_histories_watchtime["estimated_watch_hours"].sum(skipna=True)), 3)},
        {"section": "status", "measure": "cut_off_by_next_watch", "value": count_status("cut_off_by_next_watch")},
        {"section": "status", "measure": "full_duration_before_next_watch", "value":
count_status("full_duration_before_next_watch")},
        {"section": "status", "measure": "missing_duration", "value": count_status("missing_duration")},
        {"section": "status", "measure": "last_observed_event_unknown", "value":
count_status("last_observed_event_unknown")},
        {"section": "settings", "measure": "assume_full_duration_for_last_event", "value":
str(ASSUME_FULL_DURATION_FOR_LAST_EVENT)},
        {"section": "settings", "measure": "cap_estimated_watchtime_to_duration", "value":
str(CAP_ESTIMATED_WATCHTIME_TO_DURATION)},
    ]
)

watchtime_diagnostics

```

Validate And Save Outputs

The event-level output should keep exactly the same number of rows as `video_histories_enriched.csv`. Estimated watch time, when present, should be non-negative and no larger than the metadata duration.

```

if len(video_histories_watchtime) != len(video_histories_enriched):
    raise ValueError("Watch-time output changed the number of watch events.")

estimated_rows = video_histories_watchtime["estimated_watch_seconds"].notna()
if (video_histories_watchtime.loc[estimated_rows, "estimated_watch_seconds"] < 0).any():
    raise ValueError("Estimated watch time contains negative values.")

if CAP_ESTIMATED_WATCHTIME_TO_DURATION:
    too_large = (
        video_histories_watchtime.loc[estimated_rows, "estimated_watch_seconds"]
        > video_histories_watchtime.loc[estimated_rows, "duration_seconds"]
    )
    if too_large.any():
        raise ValueError("Estimated watch time exceeds metadata duration for some rows.")

video_histories_watchtime.to_csv(WATCHTIME_EVENTS_PATH, index=False)
watchtime_summary.to_csv(WATCHTIME_SUMMARY_PATH, index=False)
watchtime_diagnostics.to_csv(WATCHTIME_DIAGNOSTICS_PATH, index=False)

print(f"Saved {WATCHTIME_EVENTS_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Saved {WATCHTIME_SUMMARY_PATH.relative_to(PROJECT_ROOT).as_posix()}")
print(f"Saved {WATCHTIME_DIAGNOSTICS_PATH.relative_to(PROJECT_ROOT).as_posix()}")

```

Result

The event-level table keeps the watch-history unit of analysis and adds the watch-time estimate. The participant summary collapses those event estimates across the full mock dataset.

```
print(f"video_histories_watchtime.csv ({len(video_histories_watchtime)} rows)")
display(video_histories_watchtime.head(5))

print(f"watchtime_summary.csv ({len(watchtime_summary)} rows)")
display(watchtime_summary.head(5))

print(f"watchtime_diagnostics.csv ({len(watchtime_diagnostics)} rows)")
display(watchtime_diagnostics.head(20))
```

06 Demonstrate Enriched YouTube Exposure Measures

This notebook shows what becomes possible once donated YouTube watch histories have been cleaned, linked to scraped platform metadata, and annotated with event-level indicators.

The goal is demonstration rather than new measurement. We use the enriched watch-event table from notebook 04, plus the watch-time estimates from notebook 05 when available, to show how one watch event can now carry donation fields, metadata fields, pathway indicators, format labels, day/night labels, estimated watch time, and problematic-view labels side by side. Then we create a few compact figures that mirror the kinds of claims discussed in the article manuscript.

The notebook reads:

- `outputs/tables/video_histories_enriched.csv`
- `outputs/tables/video_histories_watchtime.csv` when available
- `outputs/tables/exposure_linkage_report.csv` when available

It writes:

- `outputs/tables/demo_enriched_event_examples.csv`
- `outputs/tables/demo_measure_summary.csv`
- `outputs/figures/demo_pathway_mix.png`
- `outputs/figures/demo_participant_measure_shares.png`
- `outputs/figures/demo_daynight_shorts.png`
- `outputs/figures/demo_problematic_views.png`

```
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

pd.set_option("display.max_columns", 120)
pd.set_option("display.max_colwidth", 90)
plt.rcParams.update({
    "figure.dpi": 120,
    "savefig.dpi": 180,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.titleweight": "bold",
})
```

Locate Project Files

The notebook can be run from the project root or from inside the `scripts` folder. All printed paths are project-relative so rendered output does not expose private local folders.

```

def find_project_root(start=None):
    start = Path.cwd() if start is None else Path(start)
    for candidate in [start, *start.parents]:
        has_quarto_project = (candidate / "_quarto.yml").exists()
        has_enriched_table = (candidate / "outputs" / "tables" / "video_histories_enriched.csv").exists()
        if has_quarto_project and has_enriched_table:
            return candidate
    raise FileNotFoundError(
        "Could not find the Quarto project root. Run this notebook from "
        "inside youtube_donation_dsa_method after notebook 04 has created the enriched table."
    )

PROJECT_ROOT = find_project_root()
OUTPUT_DIR = PROJECT_ROOT / "outputs" / "tables"
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures"

ENRICHED_VIDEO_HISTORIES_PATH = OUTPUT_DIR / "video_histories_enriched.csv"
WATCHTIME_EVENTS_PATH = OUTPUT_DIR / "video_histories_watchtime.csv"
EXPOSURE_LINKAGE_REPORT_PATH = OUTPUT_DIR / "exposure_linkage_report.csv"
DEMO_EVENT_EXAMPLES_PATH = OUTPUT_DIR / "demo_enriched_event_examples.csv"
DEMO_MEASURE_SUMMARY_PATH = OUTPUT_DIR / "demo_measure_summary.csv"

FIGURE_PATHS = {
    "pathway_mix": FIGURE_DIR / "demo_pathway_mix.png",
    "participant_measure_shares": FIGURE_DIR / "demo_participant_measure_shares.png",
    "daynight_shorts": FIGURE_DIR / "demo_daynight_shorts.png",
    "problematic_views": FIGURE_DIR / "demo_problematic_views.png",
}

print(f"Project folder: {PROJECT_ROOT.name}")
print(f"Enriched input: {ENRICHED_VIDEO_HISTORIES_PATH.relative_to(PROJECT_ROOT).as_posix()}")
if WATCHTIME_EVENTS_PATH.exists():
    print(f"Watch-time input: {WATCHTIME_EVENTS_PATH.relative_to(PROJECT_ROOT).as_posix()}")
else:
    print("Watch-time input: not found; continuing without estimated watch-time columns")
print(f"Figure folder: {FIGURE_DIR.relative_to(PROJECT_ROOT).as_posix()}")

```

Load The Enriched Event Table

The input table is event-level: each row is one watch event. The columns added by notebook 04 are treated as the source of truth here. This notebook reshapes them for display and plotting, but it does not create new classifications.

```

def ensure_columns(dataframe, required_columns, label):
    missing_columns = sorted(set(required_columns) - set(dataframe.columns))
    if missing_columns:
        raise ValueError(
            f"Missing columns in {label}: {missing_columns}. "
            f"Found: {list(dataframe.columns)}"
        )

def to_nullable_bool(series):
    normalized = series.astype("string").str.strip().str.lower()
    mapped = normalized.map(
        {
            "true": True,
            "false": False,
            "1": True,
            "0": False,
            "yes": True,
            "no": False,
        }
    )
    return mapped.astype("boolean")

video_histories_enriched = pd.read_csv(ENRICHED_VIDEO_HISTORIES_PATH, low_memory=False)
video_histories_enriched["watch_event_position"] = range(len(video_histories_enriched))

required_columns = {
    "Participant ID",
    "time",
    "watch_local_time",
    "video_id",
    "clean_title",
    "url",
    "metadata_title",
    "metadata_channel",
    "duration_seconds",
    "view_count",
    "metadata_status",
    "searched_for",
    "subscribed",
    "is_short",
    "time_of_day",
    "generalized_error_msg",
    "problematic_category",
    "problematic_assessment_label",
    "is_problematic",
}
ensure_columns(video_histories_enriched, required_columns, "video_histories_enriched.csv")

row_count = len(video_histories_enriched)
video_histories_enriched["time"] = pd.to_datetime(
    video_histories_enriched["time"],
    utc=True,
    format="mixed",
    errors="coerce",
)
video_histories_enriched["watch_local_time"] = pd.to_datetime(
    video_histories_enriched["watch_local_time"],
    format="mixed",
    errors="coerce",
)

for boolean_column in ["searched_for", "subscribed", "is_short", "is_problematic"]:
    video_histories_enriched[boolean_column] = to_nullable_bool(video_histories_enriched[boolean_column])

if video_histories_enriched["time"].isna().any():
    raise ValueError("Some rows have invalid UTC watch times.")
if len(video_histories_enriched) != row_count:
    raise AssertionError("Loading changed the number of enriched watch events.")

watchtime_available = WATCHTIME_EVENTS_PATH.exists()
if watchtime_available:
    watchtime_events = pd.read_csv(
        WATCHTIME_EVENTS_PATH,
        usecols=[
            "watch_event_position",
            "estimated_watch_seconds",
            "estimated_watch_minutes",
            "estimated_watch_hours",
            "watchtime_status",
        ],
        low_memory=False,
    )
    video_histories_enriched = video_histories_enriched.merge(

```

```

        watchtime_events,
        on="watch_event_position",
        how="left",
        validate="one_to_one",
    )
else:
    watchtime_events = pd.DataFrame()

input_summary = pd.DataFrame(
    {
        "measure": [
            "watch_events",
            "participants",
            "unique_videos",
            "earliest_watch_utc",
            "latest_watch_utc",
            "watchtime_estimates_available",
        ],
        "value": [
            row_count,
            video_histories_enriched["Participant ID"].nunique(),
            video_histories_enriched["video_id"].nunique(),
            video_histories_enriched["time"].min().isoformat(),
            video_histories_enriched["time"].max().isoformat(),
            str(watchtime_available),
        ],
    }
)

input_summary

```

Prepare Reader-Friendly Labels

The columns below are display helpers for the showcase and figures. They keep the underlying event flags unchanged, but make the notebook easier to read.

```

def shorten_text(value, max_chars=72):
    if pd.isna(value):
        return pd.NA
    text = " ".join(str(value).split())
    if len(text) <= max_chars:
        return text
    return text[: max_chars - 3].rstrip() + "..."
```

```

def status_label(value):
    if pd.isna(value):
        return "Unknown"
    if value is pd.NA:
        return "Unknown"
    return "Yes" if bool(value) else "No"
```

```

def subscribed_label(value):
    if pd.isna(value):
        return "Unknown"
    return "Yes" if bool(value) else "No"
```

```

def format_number(value):
    if pd.isna(value):
        return pd.NA
    return f"{int(value):,}"
```

```

analysis_events = video_histories_enriched.copy()
analysis_events["searched_confirmed"] = analysis_events["searched_for"].eq(True)
analysis_events["subscribed_confirmed"] = analysis_events["subscribed"].eq(True)
analysis_events["short_confirmed"] = analysis_events["is_short"].eq(True)
analysis_events["problematic_confirmed"] = analysis_events["is_problematic"].eq(True)
analysis_events["night_confirmed"] = analysis_events["time_of_day"].astype("string").str.lower().eq("night")
analysis_events["video_format"] = "Longs"
analysis_events.loc[analysis_events["short_confirmed"], "video_format"] = "Shorts"
analysis_events["time_period"] = analysis_events["time_of_day"].astype("string").str.title()
analysis_events["pathway"] = "Neither search nor subscription"
analysis_events.loc[
    analysis_events["searched_confirmed"] & ~analysis_events["subscribed_confirmed"],
    "pathway",
] = "Search-associated only"
analysis_events.loc[
    ~analysis_events["searched_confirmed"] & analysis_events["subscribed_confirmed"],
    "pathway",
] = "Subscribed-channel only"
analysis_events.loc[
    analysis_events["searched_confirmed"] & analysis_events["subscribed_confirmed"],
    "pathway",
] = "Both search and subscription"

analysis_events["clean_title_short"] = analysis_events["clean_title"].map(shorten_text)
analysis_events["metadata_title_short"] = analysis_events["metadata_title"].map(shorten_text)
analysis_events["generalized_error_msg_short"] = analysis_events["generalized_error_msg"].map(shorten_text)
analysis_events["duration_minutes"] = (pd.to_numeric(analysis_events["duration_seconds"], errors="coerce") / 60).round(1)
if "estimated_watch_minutes" in analysis_events.columns:
    analysis_events["estimated_watch_minutes"] = pd.to_numeric(
        analysis_events["estimated_watch_minutes"], errors="coerce"
    ).round(1)
analysis_events["view_count_display"] = pd.to_numeric(analysis_events["view_count"], errors="coerce").map(format_number)
analysis_events[["pathway", "video_format", "time_period"]].head()
```

What The Enriched Data Now Makes Visible

The table below is a small set of illustrative watch events. It is meant to show the shape of the enriched data before we aggregate anything: a donated watch event can now be read together with scraped metadata, pathway context, format labels, day/night timing, and problematic-view signals.

The participant names are safe here because the data are mock data. In a real donation study, this section should be pseudonymized, aggregated, or suppressed if small cells could reveal sensitive viewing behavior.

```

example_specs = [
    ("Search-associated watch", analysis_events["searched_confirmed"]),
    ("Subscribed-channel watch", analysis_events["subscribed_confirmed"]),
    ("Shorts watch", analysis_events["short_confirmed"]),
    ("Nighttime watch", analysis_events["night_confirmed"]),
    ("Problematic/unavailable signal", analysis_events["problematic_confirmed"]),
    (
        "Neither search nor subscription",
        analysis_events["pathway"].eq("Neither search nor subscription")
        & ~analysis_events["problematic_confirmed"],
    ),
]

selected_rows = []
used_indices = set()
for reason, mask in example_specs:
    candidates = analysis_events[mask].copy()
    if candidates.empty:
        continue
    unused_candidates = candidates.loc[~candidates.index.isin(used_indices)]
    chosen = unused_candidates.head(1) if not unused_candidates.empty else candidates.head(1)
    chosen = chosen.copy()
    chosen["showcase_reason"] = reason
    selected_rows.append(chosen)
    used_indices.add(chosen.index[0])

if not selected_rows:
    raise ValueError("No showcase rows could be selected from the enriched table.")

showcase_columns = [
    "showcase_reason",
    "Participant ID",
    "watch_local_time",
    "clean_title_short",
    "url",
    "metadata_title_short",
    "metadata_channel",
    "duration_minutes",
    "view_count_display",
    "searched_for",
    "subscribed",
    "video_format",
    "time_period",
    "metadata_status",
    "generalized_error_msg_short",
    "problematic_assessment_label",
]

if "estimated_watch_minutes" in analysis_events.columns:
    insert_after = showcase_columns.index("duration_minutes") + 1
    showcase_columns.insert(insert_after, "estimated_watch_minutes")

demo_enriched_event_examples = pd.concat(selected_rows, ignore_index=True)[showcase_columns].rename(
    columns={
        "Participant ID": "participant_id",
        "watch_local_time": "watch_local_time",
        "clean_title_short": "clean_title",
        "metadata_title_short": "metadata_title",
        "metadata_channel": "channel",
        "duration_minutes": "duration_minutes",
        "estimated_watch_minutes": "estimated_watch_minutes",
        "view_count_display": "view_count",
        "searched_for": "searched",
        "subscribed": "subscribed",
        "generalized_error_msg_short": "generalized_error_msg",
    }
)

demo_enriched_event_examples["searched"] = demo_enriched_event_examples["searched"].map(status_label)
demo_enriched_event_examples["subscribed"] = demo_enriched_event_examples["subscribed"].map(subscribed_label)
demo_enriched_event_examples["watch_local_time"] = demo_enriched_event_examples["watch_local_time"].astype("string")

demo_enriched_event_examples

```

Build Compact Summary Tables

These tables turn the event-level columns into aggregate measures. The goal is to make the denominator visible: the shares below are shares of watch events in the enriched table.

```

watch_events = len(analysis_events)

def event_count_share(label, count):
    return {
        "section": "whole_dataset",
        "measure": label,
        "value": int(count),
        "share_of_watch_events": count / watch_events if watch_events else pd.NA,
    }

summary_rows = [
    {"section": "whole_dataset", "measure": "watch_events", "value": watch_events, "share_of_watch_events": pd.NA},
    {"section": "whole_dataset", "measure": "participants", "value": analysis_events["Participant ID"].nunique(),
    "share_of_watch_events": pd.NA},
    {"section": "whole_dataset", "measure": "unique_videos", "value": analysis_events["video_id"].nunique(),
    "share_of_watch_events": pd.NA},
    event_count_share("searched_watch_events", analysis_events["searched_confirmed"].sum()),
    event_count_share("confirmed_subscribed_watch_events", analysis_events["subscribed_confirmed"].sum()),
    event_count_share("shorts_watch_events", analysis_events["short_confirmed"].sum()),
    event_count_share("nighttime_watch_events", analysis_events["night_confirmed"].sum()),
    event_count_share("problematic_watch_events", analysis_events["problematic_confirmed"].sum()),
]

if "estimated_watch_hours" in analysis_events.columns:
    summary_rows.append({
        "section": "whole_dataset",
        "measure": "total_estimated_watch_hours",
        "value": round(float(pd.to_numeric(analysis_events["estimated_watch_hours"], errors="coerce").sum(skipna=True)),
3),
        "share_of_watch_events": pd.NA,
    })

demo_measure_summary = pd.DataFrame(summary_rows)

participant_aggregations = dict(
    watch_events=("video_id", "size"),
    searched_events=("searched_confirmed", "sum"),
    subscribed_events=("subscribed_confirmed", "sum"),
    shorts_events=("short_confirmed", "sum"),
    nighttime_events=("night_confirmed", "sum"),
    problematic_events=("problematic_confirmed", "sum"),
)

if "estimated_watch_hours" in analysis_events.columns:
    analysis_events["estimated_watch_hours"] = pd.to_numeric(analysis_events["estimated_watch_hours"], errors="coerce")
    participant_aggregations["estimated_watch_hours"] = ("estimated_watch_hours", "sum")

participant_summary = (
    analysis_events.groupby("Participant ID", dropna=False)
    .agg(**participant_aggregations)
    .reset_index()
    .rename(columns={"Participant ID": "participant_id"})
)

for count_column, share_column in [
    ("searched_events", "searched_share"),
    ("subscribed_events", "subscribed_share"),
    ("shorts_events", "shorts_share"),
    ("nighttime_events", "nighttime_share"),
    ("problematic_events", "problematic_share"),
]:
    participant_summary[share_column] = participant_summary[count_column] / participant_summary["watch_events"]

pathway_order = [
    "Neither search nor subscription",
    "Search-associated only",
    "Subscribed-channel only",
    "Both search and subscription",
]

pathway_summary = (
    analysis_events["pathway"]
    .value_counts()
    .reindex(pathway_order, fill_value=0)
    .rename_axis("pathway")
    .reset_index(name="watch_events")
)

pathway_summary["share_of_watch_events"] = pathway_summary["watch_events"] / watch_events

format_order = ["Shorts", "Longs"]
time_order = ["Day", "Night"]
daynight_format_summary = pd.crosstab(
    analysis_events["time_period"],
    analysis_events["video_format"],
).reindex(index=time_order, columns=format_order, fill_value=0)

```

```

problematic_assessment_summary = (
    analysis_events["problematic_assessment_label"]
    .value_counts(dropna=False)
    .rename_axis("problematic_assessment_label")
    .reset_index(name="watch_events")
)
problematic_assessment_summary["share_of_watch_events"] = problematic_assessment_summary["watch_events"] / watch_events

problematic_category_summary = (
    analysis_events["problematic_category"]
    .value_counts(dropna=False)
    .rename_axis("problematic_category")
    .reset_index(name="watch_events")
)
problematic_category_summary["share_of_watch_events"] = problematic_category_summary["watch_events"] / watch_events

share_columns = [column for column in participant_summary.columns if column.endswith("_share")]
if not participant_summary[share_columns].map(lambda value: 0 <= value <= 1).all().all():
    raise AssertionError("Participant-level shares should be between 0 and 1.")

if len(analysis_events) != row_count:
    raise AssertionError("Summary construction changed the enriched input row count.")

demo_measure_summary

```

Figure 1: Pathway Mix

This figure illustrates the manuscript's pathway question: how much watched content appears to be associated with search, with subscribed channels, with both, or with neither observed route? The categories are indicators, not causal explanations of how YouTube produced the view.

```

FIGURE_DIR.mkdir(parents=True, exist_ok=True)

plot_data = pathway_summary.sort_values("watch_events", ascending=True)
fig, ax = plt.subplots(figsize=(8, 4.6))
colors = ["#6b7280", "#2f6f9f", "#c17c2c", "#4f8f5b"]
ax.barh(plot_data["pathway"], plot_data["watch_events"], color=colors[: len(plot_data)])
ax.set_title("Watch Events By Observed Pathway")
ax.set_xlabel("Watch events")
ax.set_ylabel("")
for index, row in plot_data.reset_index(drop=True).iterrows():
    ax.text(
        row["watch_events"] + 2,
        index,
        f"{row['watch_events']} ({row['share_of_watch_events']:.0%})",
        va="center",
        fontsize=9,
    )
ax.set_xlim(0, max(plot_data["watch_events"]) * 1.2)
fig.tight_layout()
fig.savefig(FIGURE_PATHS["pathway_mix"], bbox_inches="tight")
plt.show()

pathway_summary

```

Figure 2: Participant-Level Measure Shares

The same event-level flags can also be summarized by participant. In the mock data this is safe to display; with real donations, participant-level outputs should be pseudonymized and checked for disclosure risk.

```

measure_specs = [
    ("searched_share", "Searched"),
    ("subscribed_share", "Subscribed"),
    ("shorts_share", "Shorts"),
    ("nighttime_share", "Night"),
    ("problematic_share", "Problematic"),
]
participants = participant_summary["participant_id"].tolist()
x_positions = list(range(len(participants)))
bar_width = 0.15
palette = ["#2f6f9f", "#c17c2c", "#4f8f5b", "#7f6a93", "#b4473a"]

fig, ax = plt.subplots(figsize=(9, 4.8))
for measure_index, (column, label) in enumerate(measure_specs):
    offset = (measure_index - (len(measure_specs) - 1) / 2) * bar_width
    positions = [x + offset for x in x_positions]
    ax.bar(
        positions,
        participant_summary[column],
        width=bar_width,
        label=label,
        color=palette[measure_index],
    )

ax.set_title("Selected Exposure Measures By Mock Participant")
ax.set_ylabel("Share of watch events")
ax.set_xticks(x_positions)
ax.set_xticklabels(participants)
ax.set_ylim(0, max(0.3, participant_summary[[column for column, _ in measure_specs]].max().max() * 1.25))
ax.yaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
ax.legend(ncol=3, frameon=False, loc="upper center", bbox_to_anchor=(0.5, -0.12))
fig.tight_layout()
fig.savefig(FIGURE_PATHS["participant_measure_shares"], bbox_inches="tight")
plt.show()

participant_summary

```

Figure 3: Day/Night And Shorts/Longs

This figure combines the locally interpreted watch time with metadata-derived video format. It shows how routine-oriented questions can be crossed with content-format indicators once timestamps and metadata sit in the same event table.

```

fig, ax = plt.subplots(figsize=(7, 4.6))
bottom_values = [0] * len(daynight_format_summary)
format_colors = {"Shorts": "#4f8f5b", "Longs": "#6b7280"}

for format_label in format_order:
    values = daynight_format_summary[format_label].tolist()
    ax.bar(
        daynight_format_summary.index,
        values,
        bottom=bottom_values,
        label=format_label,
        color=format_colors[format_label],
    )
    bottom_values = [bottom + value for bottom, value in zip(bottom_values, values)]

ax.set_title("Day And Night Watching By Video Format")
ax.set_ylabel("Watch events")
ax.set_xlabel("")
ax.legend(frameon=False)
for index, total in enumerate(bottom_values):
    ax.text(index, total + 3, str(total), ha="center", fontsize=9)
ax.set_ylim(0, max(bottom_values) * 1.18)
fig.tight_layout()
fig.savefig(FIGURE_PATHS["daynight_shorts"], bbox_inches="tight")
plt.show()

daynight_format_summary

```

Figure 4: Problematic And Unavailable Signals

The problematic-view labels come from metadata errors and the manual classification table used in notebook 04. They should be read as later availability and classification signals, not as a direct content analysis of what the participant saw at the time of watching.

```
plot_data = problematic_assessment_summary.sort_values("watch_events", ascending=True)
fig, ax = plt.subplots(figsize=(8, 4.6))
assessment_colors = ["#6b7280", "#2f6f9f", "#b4473a", "#c17c2c"]
ax.barh(
    plot_data["problematic_assessment_label"],
    plot_data["watch_events"],
    color=assessment_colors[: len(plot_data)],
)
ax.set_title("Watch Events By Availability / Problematic Assessment")
ax.set_xlabel("Watch events")
ax.set_ylabel("")
for index, row in plot_data.reset_index(drop=True).iterrows():
    ax.text(
        row["watch_events"] + 2,
        index,
        f"{row['watch_events']} ({row['share_of_watch_events']:.0%})",
        va="center",
        fontsize=9,
    )
ax.set_xlim(0, max(plot_data["watch_events"]) * 1.2)
fig.tight_layout()
fig.savefig(FIGURE_PATHS["problematic_views"], bbox_inches="tight")
plt.show()

problematic_category_summary
```

Save Demo Outputs

The CSV outputs keep the compact showcase table and whole-dataset summary. The figures are saved as standalone PNG files so they can be used in the manuscript, slides, or a rendered Quarto document.

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

demo_enriched_event_examples.to_csv(DEMO_EVENT_EXAMPLES_PATH, index=False)
demo_measure_summary.to_csv(DEMO_MEASURE_SUMMARY_PATH, index=False)

for path in [DEMO_EVENT_EXAMPLES_PATH, DEMO_MEASURE_SUMMARY_PATH, *FIGURE_PATHS.values()]:
    if not path.exists():
        raise AssertionError(f"Expected output was not created: {path.name}")

saved_outputs = pd.DataFrame(
    {
        "output": [
            DEMO_EVENT_EXAMPLES_PATH.relative_to(PROJECT_ROOT).as_posix(),
            DEMO_MEASURE_SUMMARY_PATH.relative_to(PROJECT_ROOT).as_posix(),
            *[path.relative_to(PROJECT_ROOT).as_posix() for path in FIGURE_PATHS.values()],
        ]
    }
)

saved_outputs
```

Result

The notebook stores a compact showcase table, a whole-dataset summary table, and four figures. The previews below are the main handoff for the article-facing demonstration.

```
print("demo_enriched_event_examples.csv")
display(demo_enriched_event_examples.head(10))

print("demo_measure_summary.csv")
display(demo_measure_summary.head(10))

print("Participant summary")
display(participant_summary)

print("Pathway summary")
display(pathway_summary)

print("Problematic assessment summary")
display(problematic_assessment_summary)

print("Saved outputs")
for path in [DEMO_EVENT_EXAMPLES_PATH, DEMO_MEASURE_SUMMARY_PATH, *FIGURE_PATHS.values()]:
    print(path.relative_to(PROJECT_ROOT).as_posix())
```